

DEEP LEARNING, TOLD SIMPLY

Book 6 of 6

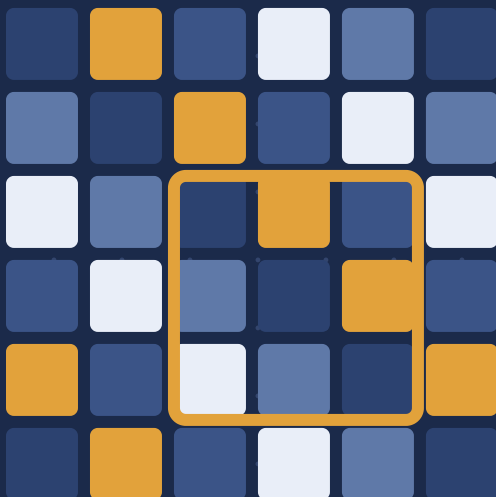
WRITTEN BY

Prof. Saurabh

Eyes for the Machine

Convolution, feature maps, pooling, the famous CNNs, and how to borrow eyes that are already trained

Asha gives Nova a camera. Nova must recognise Asha's face, and the cat on the hostel wall, wherever they appear in the picture. A plain network needs one weight for every pixel and fails as soon as the face moves. So Asha discovers three simple ideas — look locally, reuse the same detector everywhere, shrink — and builds a convolutional network by hand. This book follows Module 6 of the Deep Neural Network course, slide by slide, in plain words.



A companion to the Deep Neural Network course · Module 6



Asha & Nova · 2026

What is inside

Book 6 covers Module 6: Convolutional Neural Networks. Chapters 3 to 6 do convolution, pooling and the final layers by hand with the deck's own numbers; 7 to 9 cover design, the famous networks, and practice.

1	What a camera gives Nova	3
	A picture is a grid of numbers; the hard part is finding "Asha" in them, anywhere.	
2	Three ideas that make eyes cheap	8
	Look locally, share one detector everywhere, shrink: far fewer weights than a plain network.	
3	Convolution by hand	14
	Slide a small filter over the picture, multiply and add at every spot: a feature map.	
4	How far a neuron can see	20
	Stacked layers see further; padding, stride and dilation control the geometry.	
5	Depth, shrinking, and switching on	27
	Filters have depth, pooling shrinks for free, 1×1 mixes channels, ReLU switches off misses.	
6	The head that decides	34
	Flatten, score with a fully connected layer, turn scores into probabilities with softmax.	
7	Counting knobs and designing eyes	38
	Count a layer's parameters, then design with five patterns and a few rules of thumb.	
8	The famous ones	43
	LeNet, AlexNet, VGG, NiN, GoogLeNet and ResNet: six networks, six ideas.	
9	Borrowing eyes, and using them	51
	Borrow eyes trained on ImageNet, then see what CNNs are used for and how to make them work.	

How to read this book

Each chapter follows the same rhythm: a little story, one big idea, an everyday picture of it, the maths in small steps (first plain, then in matrix form), a worked example with real numbers, and a short check. The coloured boxes tell you what kind of thing you are reading:

- | | |
|--|--|
|  Story — Asha and Nova, the thread through all six books |  Big idea — the one thing to remember |
|  Think of it like... — a real-life picture of the idea |  Stop and think — a question, then Our answer |
|  Maths, step by step — plain maths → matrix form |  Worked example — real numbers, one line per step |
|  On your slides — the exact notation your deck uses |  Careful — common mistakes |
|  Spark — a deeper idea for the curious |  Check yourself — quick questions with answers |

What a camera gives Nova

A picture is a grid of numbers; the hard part is finding "Asha" in them, anywhere.

On your slides: **Module 6, slides 4–16** · Computer vision tasks · Grayscale and colour images · Challenges: semantic gap, size, invariance, equivariance · The solution

STORY

Nova can hear its name now. Tonight Asha screws a small camera onto Nova's head. "Look at me," she says, and Nova's tiny screen fills with numbers, hundreds of them, scrolling past. Ravi leans in the doorway. "That is your face?" "That is what the camera gives it," says Asha. "A wall of numbers. And tomorrow, when I sit on the other side of the room, it will be a completely different wall of numbers, and it will still be my face."

BIG IDEA

To a computer, a picture is only a grid of numbers: one number per pixel for a grey picture, three per pixel for a colour picture. There are a lot of them, and the object we care about can sit anywhere in the grid. A good vision network must find **patterns** in the numbers, and find them **wherever they are**.

THINK OF IT LIKE...

A class register with 2,000 students, one mark per student. Now someone asks "is this a good class?" No single mark answers that. You must look at groups of marks and notice patterns. And the pattern "a strong group" is the same pattern whether it appears at the top of the register or at the bottom. A picture is the same: no single pixel says "face", and a face is a face at the top-left or the bottom-right.

Four jobs a computer does with pictures

Your slides open with four pictures. Each one is a different job, and each is an input → output mapping, like every task in Book 1.

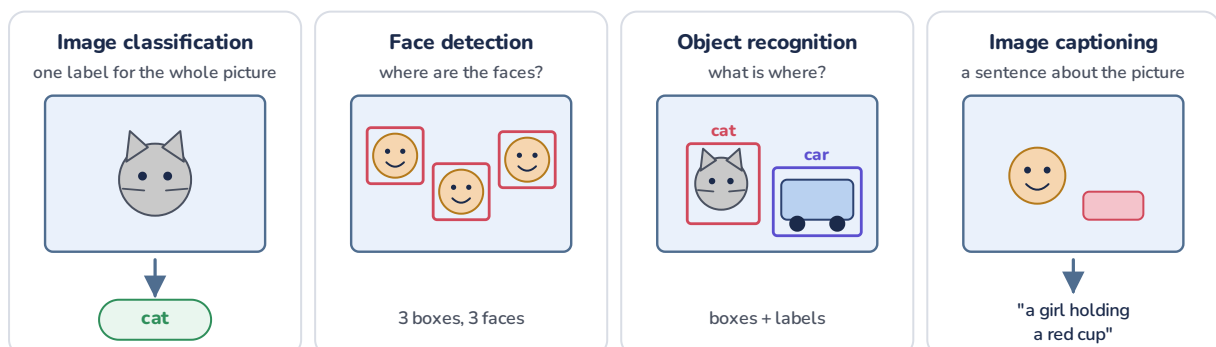


Figure 1.1 The four tasks on your first slides. **Classification:** the whole picture gets one label. **Face detection:** find every face and draw a box around it. **Object recognition:** find every object, box it, and name it. **Captioning:** write a sentence about the picture. Notice that the input is always the same kind of thing, a picture; only the output changes.

This book is mostly about the first job, classification, because the other three are built on top of it. Nova's job is classification: picture in, "Asha / cat / neither" out.

A picture is a grid of numbers

A **grayscale** (black-and-white) picture is a grid. Each little square in the grid is a **pixel**. Each pixel holds one number that says how bright it is: 0 is black, 255 is white, and the numbers in between are shades of grey. Your slide writes the brightness at column x , row y as $I(x, y)$. The letter I stands for the image.



Figure 1.2 An 8×8 grey picture as the computer stores it: one number per pixel. The bright stroke of the "7" is the run of big numbers (240, 250). Nothing "sees" a seven; the computer only holds 64 numbers.

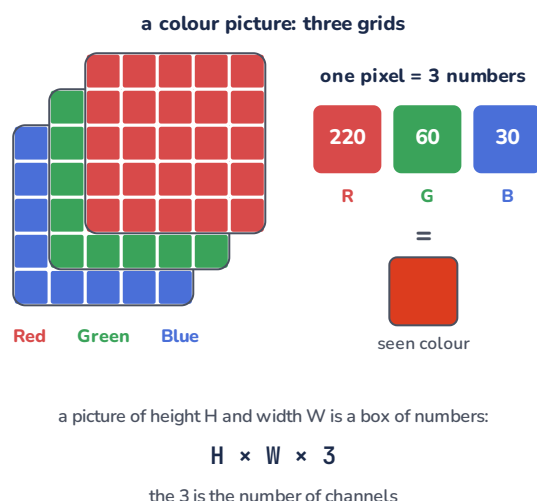


Figure 1.3 A colour picture is three grids, one per channel: red, green, blue. One pixel is three numbers, and a picture of height H and width W is a box of $H \times W \times 3$ numbers. That " $\times 3$ " becomes the depth of the first filter in Chapter 5.

A **colour** picture is three such grids stacked together: one for red, one for green, one for blue. Each pixel now has three numbers. The three grids are called **channels**.

Σ MATHS, STEP BY STEP

PLAIN MATHS

One pixel of a grey picture is one number:

$$I(x, y) \in \{0, 1, \dots, 255\}$$

x is the column, y is the row. For Figure 1.2, $I(2, 1) = 250$: second row, third column (counting from 0).

One pixel of a colour picture is three numbers:

$$I(x, y, c), \quad c \in \{R, G, B\}$$

c picks the channel.

MATRIX FORM

A grey picture is one matrix:

$$\mathbf{I} \in \mathbb{R}^{H \times W}$$

H rows, W columns. Figure 1.2 is 8×8 .

A colour picture is a 3-D box of numbers, a **tensor**:

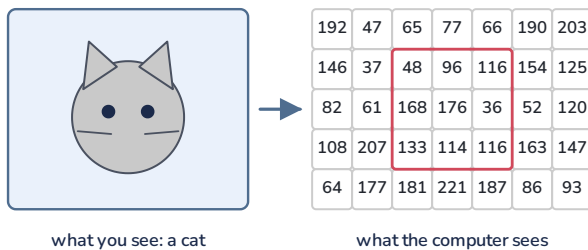
$$\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$$

A network will treat the whole box at once. The number of values in it is $H \cdot W \cdot 3$. That count is the subject of the next section.

Why this is hard

1. THE SEMANTIC GAP

You see a cat. The computer holds a grid of numbers between 0 and 255. The distance between "these numbers" and "this is a cat" is what your slides call the **semantic gap**. Nothing about the number 147 says "fur".



the slide's picture: $800 \times 600 \times 3 = 1,440,000$ numbers

Nothing about the number 147 says "fur".

The distance between the numbers and the word "cat" is the semantic gap.

Figure 1.4 The slide's cat and what the computer sees: a grid of numbers, 1,440,000 of them for an 800×600 colour picture. That distance is the semantic gap.

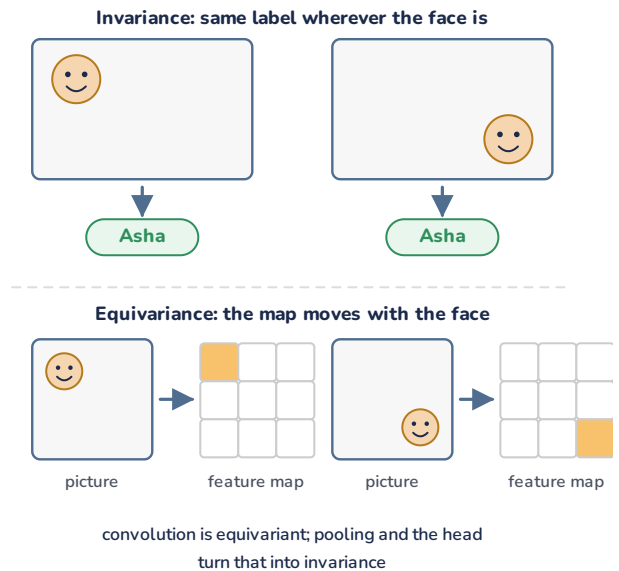


Figure 1.5 Top, invariance: the label must not change when the face moves. Bottom, equivariance: the network's internal map has a bright spot that moves with the face. Convolution gives equivariance; pooling and the final layers turn it into invariance.

2. THERE ARE A LOT OF NUMBERS

WORKED EXAMPLE: HOW MANY INPUTS?

Your slide gives two sizes. Let us count the inputs a network would receive.

- 1 A small 50×50 grey picture: $50 \times 50 = 2500$ numbers.
one number per pixel
- 2 An HD picture, 1920×1080 : $1920 \times 1080 = 2,073,600$ numbers, about 2 million.
the slide says "about 2 million inputs (2 megapixels)"; with colour it is three times more
- 3 Suppose we feed the small picture to a fully connected layer with just 100 neurons, the kind of layer from Book 5. Each neuron needs one weight per input: $2500 \times 100 = 250,000$ weights.
and that is only the first layer of a tiny picture

For the HD picture the same layer would need $2,073,600 \times 100 \approx 207$ million weights. Book 5's fully connected layers do not scale to pictures. We need a cheaper kind of layer.

3. THE OBJECT CAN BE ANYWHERE: INVARIANCE

Asha's face may be at the top-left of the picture today and at the bottom-right tomorrow. The pixel numbers are then completely different, but the answer must be the same: "Asha". When the output does *not* change although the object moved, we say the network is **invariant** to the move. Your slide: "object can appear anywhere in the image; model predicts cat for both images".

4. A SHIFT IN SHOULD GIVE A SHIFT OUT: EQUIVARIANCE

Inside the network, though, we want something slightly different. If the face moves 10 pixels to the right, we want the network's internal picture of "where the face-like pattern is" to move 10 pixels to the right too. Change the input in a predictable way, and the output changes in the same predictable way. This is called **equivariance**. Your slide says it as "changes to the input predictably should not affect the output" — the map moves along with the object, so nothing is lost.

? STOP AND THINK

Ravi says: "Just save one photo of you. When a new picture comes, compare it pixel by pixel. If the numbers are close, it is you."

✓ OUR ANSWER

It fails the moment anything moves. Shift the face by five pixels and every number at every position changes; the pixel-by-pixel difference becomes huge although it is the same face. Dim the light and every number drops. Pixel matching is not invariant. We need to compare *patterns* (an eye, a nose, an edge) not *positions*. That is exactly what the rest of this book builds.

The solution

Older computer vision split the job into two parts. First, people wrote code by hand to pull out **features** from the picture: edges, corners, colour histograms. Then a simple classifier (an SVM, or a shallow network) learned to map those features to "cat / not cat". Only the second part learned. The first part was fixed by hand, and it was usually the weak link.

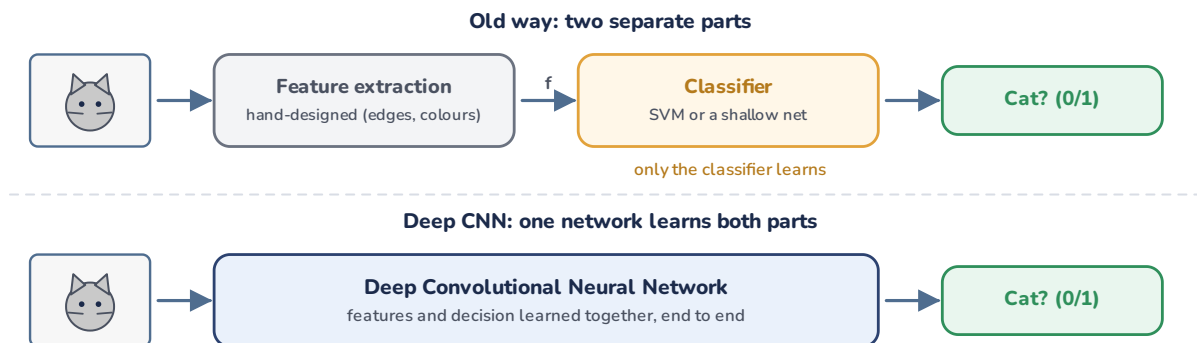


Figure 1.6 Your "solution" slide. Top: hand-made feature extraction, then a classifier that learns. Bottom: a deep convolutional network learns the features *and* the classifier together, end to end. The next chapter explains why "convolutional" is the word that makes this affordable.

A **deep convolutional neural network (CNN)** does both jobs, and learns both from data. The early layers learn to extract features. The last layers learn to classify. Nothing is written by hand except the shape of the network.

📖 ON YOUR SLIDES

Slides 4–7: the four tasks. Slide 9: the grayscale digit, "pixel intensities = $I(x, y)$ ". Slide 10: the colour image. Slides 11–13: the challenges (semantic gap, "a big grid of numbers in $[0, 255]$, e.g. $800 \times 600 \times 3$ "; $50 \times 50 = 2500$ inputs; $HD \approx 2$ million). Slide 14: invariance. Slide 15: equivariance. Slide 16: the two-pipeline solution.

! CAREFUL

Do not mix up the two words. **Invariance**: the output stays the same when the input shifts (the label). **Equivariance**: the output shifts in the same way as the input (the feature map). A CNN is equivariant inside and invariant at the end.

✓ CHECK YOURSELF

1. What does $I(x, y)$ mean? **The brightness of the pixel at column x , row y , a number from 0 to 255.**
2. How many numbers describe a colour picture of height H and width W ? **$H \times W \times 3$.**
3. How many inputs does a 50×50 grey picture give? **2500.**
4. The face moves and the label stays "Asha". Invariance or equivariance? **Invariance.**
5. In the old pipeline, which part learned? **Only the classifier; the feature extraction was hand-made.**

🕒 IN ONE BREATH

- A picture is a grid of numbers, 0 to 255. Colour means three grids: $H \times W \times 3$.
- There are millions of numbers, and no single one means "cat": the semantic gap.
- The object can be anywhere. We want invariance at the output and equivariance inside.
- Old way: hand-made features + a classifier. New way: a deep CNN learns both.

Three ideas that make eyes cheap

Look locally, share one detector everywhere, shrink: far fewer weights than a plain network.

On your slides: **Module 6, slides 18–35** · What is a CNN · Why CNNs · Locality, translation, scale · The whole CNN · Key principles and characteristics · Hierarchical features · Architecture pattern · Layer-by-layer · Classification process

📖 STORY

Asha tries the obvious thing first. She shrinks the camera picture to 50×50 , flattens it into 2,500 numbers, and feeds it to a Book 5 network. The first layer alone has 250,000 weights. It trains slowly, and then it fails: it recognises her face only in the exact spot where she sat during training. "Your network learned *where* your face was," he says, "not *what* it looks like." Asha needs a layer that looks for a face-like pattern anywhere, and does not need a separate weight for every pixel.

! BIG IDEA

A **convolutional neural network** (CNN) is built on three ideas. (1) A pattern is much smaller than the picture, so a neuron only needs to look at a small patch. (2) The same pattern can appear anywhere, so one small detector is **slid over the whole picture** and its weights are **shared**. (3) Shrinking the picture does not destroy the object, so we can **pool** (sub-sample) between layers. Together they cut the number of weights from millions to a few hundred per detector.

≈ THINK OF IT LIKE...

Searching a big poster in the dark with a small torch. The torch lights only a small patch at a time, so you look at a patch, not the whole poster: that is *locality*. You use the same eyes and the same idea of what the word "Nova" looks like at every patch; you do not learn a new way of reading for each corner: that is *parameter sharing*. When you are done, you do not remember every letter at every position; you remember "the word is near the top": that is *pooling*.

What a CNN is, and why

Your slide defines it: a CNN is a special kind of deep neural network designed for data laid out on a **grid**, such as images, where the **position** of things relative to each other matters. The core insight copies the visual part of the brain: early layers detect simple patterns (edges, corners) and deeper layers combine them into complex objects (faces, cars).

Why CNNs for images? Your slide gives five reasons. Each one is a promise that the rest of this book keeps.

- **Translation invariance:** an object is recognised no matter where it sits in the picture.
- **Local patterns:** edges, textures and shapes are found in small patches, not across the whole picture.

- **Hierarchical features:** low-level $\rightarrow$ mid-level $\rightarrow$ high-level. Edges become parts, parts become objects.
- **Parameter efficiency:** far fewer weights than a fully connected network on the same picture.
- **Computational efficiency:** the same small operation is repeated everywhere, so it runs in parallel on a GPU.

Property 1 and 2: patterns are small, and they can appear anywhere

Your slide shows a bird and asks: to find the beak, does a neuron have to see the whole picture? No. The beak sits in a small patch. So a neuron can connect to a small region only and look for one small pattern there. Your slide: "a neuron does not have to see the whole image to discover the pattern". That is **locality**.

Now a second beak, in the bottom-right. It is the same pattern. Your slide says the two detectors "do almost the same thing; they can use the same set of parameters". So we do not build two detectors. We build one and slide it over every position in the picture. One set of 9 weights serves the whole picture. This is **parameter sharing**, and sliding one filter over the whole picture is the operation called **convolution** (Chapter 3).

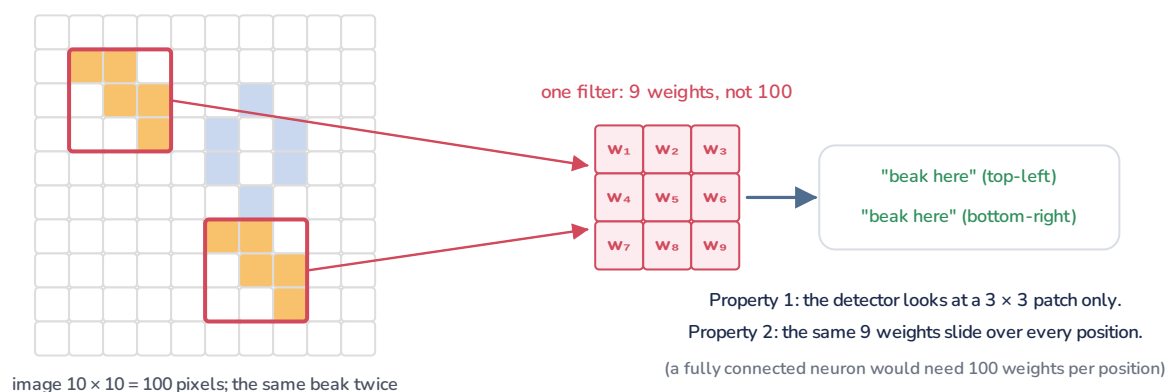


Figure 2.1 One filter with 9 weights $w_1 \dots w_9$ looks at a 3×3 patch (locality: 9 weights, not 100 for this 10×10 picture). The same 9 weights are applied at the top-left and again at the bottom-right (sharing), so the same pattern is found in both places. The small region a neuron looks at is called its receptive field (Chapter 4).

Property 3: shrinking does not change the object (scale)

Take every second pixel of a picture and throw the rest away. The bird is still a bird, just smaller. Your slide calls this **sub-sampling**: "we can sub-sample the image to make it smaller; fewer parameters for the network to process the image." This is the idea behind **pooling** (Chapter 5): shrink the maps between layers so that later layers have less to look at, and so that small shifts stop mattering.

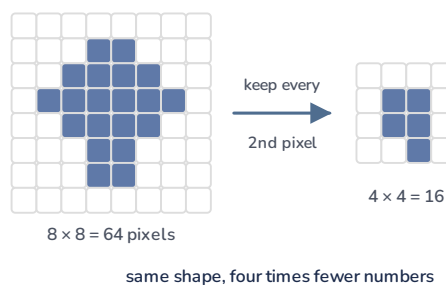


Figure 2.2 Keeping every second pixel turns 64 numbers into 16, and the shape is still there.

The whole CNN

Your slides 24–27 build the full picture in four steps. Put the three properties together and you get the standard recipe: a convolution layer (property 1 and 2), then max pooling (property 3), then convolution again, then pooling again, as many times as you like. At the end, **flatten** the last small maps into one long list and feed a fully connected network, exactly the kind from Book 5, to make the decision.

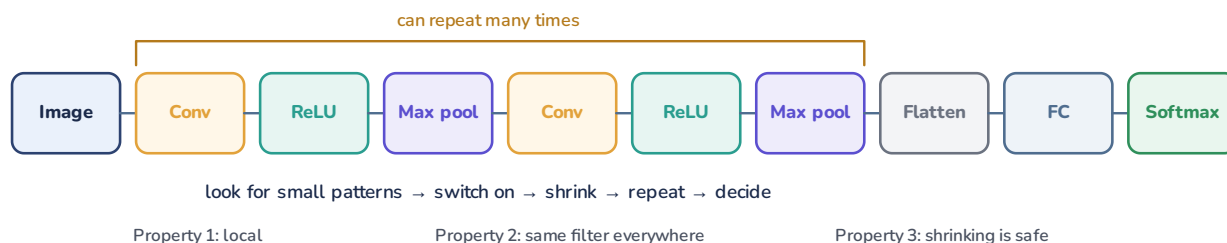


Figure 2.3 The whole CNN from your slides. Conv → ReLU → Max pool, repeated, then Flatten → FC → Softmax. The conv layers look for small patterns everywhere; ReLU switches the weak matches off; pooling shrinks; the fully connected layers decide. The bracket says "can repeat many times".

Three key principles, and the key characteristics

Your slide 28 gives the three properties their formal names. Slide 29 lists the characteristics of a CNN. Here they are side by side.

Principle (slide 28)	What it means	Characteristic (slide 29)
Local connectivity (sparse interactions)	Each neuron connects only to a local region of the input: its receptive field. (Property 1.)	Sparse connectivity; conv layers apply learnable filters to detect local features.
Parameter sharing	The same filter is used across the entire input; this cuts parameters dramatically. (Property 2.)	Parameter sharing: the same filter applied across the entire input.
Equivariant representations	If the input shifts, the output shifts in the same way: translation equivariance is built in. (Property 2.)	Pooling layers reduce spatial size and give translation invariance (Property 3); hierarchical learning builds complex features from simple ones.

Features grow more complex layer by layer

Because each layer looks at a window of the layer below, patterns can be built up in steps. Your slide gives the face example: layer 1 detects edges at various angles; layer 2 combines edges into facial features (eyes, nose, mouth); layer 3 combines those into a complete face.

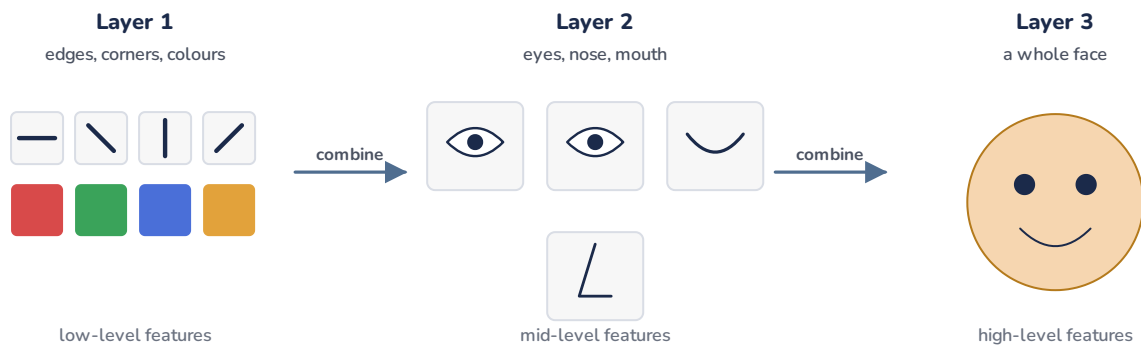


Figure 2.4 Hierarchical feature learning. Low-level features (edges, corners, colours) → mid-level (textures, patterns, parts) → high-level (objects, faces, scenes). No one tells layer 2 to look for eyes. It finds that eyes are useful because they help the final answer. This is Book 1's "layers of representation", now for pictures.

The architecture pattern, with sizes

Your slide 32 writes the recipe as a formula and gives one concrete example with all the sizes.

$$\text{INPUT} \rightarrow [\text{CONV} \rightarrow \text{ACTIVATION} \rightarrow \text{POOL}] \times N \rightarrow \text{FC} \rightarrow \text{OUTPUT}$$

The $\times N$ means the bracket is repeated N times. The example is a 32×32 colour picture going through two rounds:

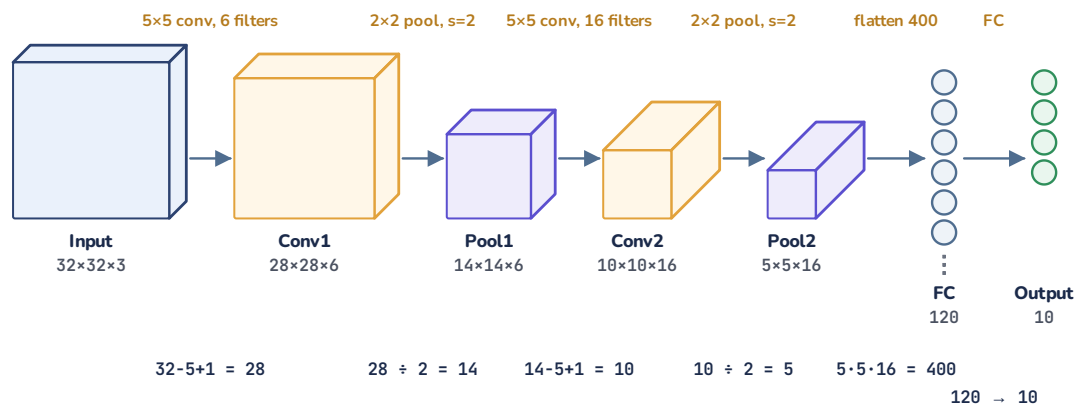


Figure 2.5 The slide's example: $32 \times 32 \times 3 \rightarrow 28 \times 28 \times 6 \rightarrow 14 \times 14 \times 6 \rightarrow 10 \times 10 \times 16 \rightarrow 5 \times 5 \times 16 \rightarrow 120 \rightarrow 10$. Each box is a stack of maps: height $\times$ width $\times$ number of maps. Under each arrow is the arithmetic that produces the next size. Notice the shape of the story: the maps get smaller ($32 \rightarrow 5$) and deeper ($3 \rightarrow 16$), then everything is flattened into a list.

WORKED EXAMPLE: WHERE DOES EACH SIZE COME FROM?

Two rules are enough. A convolution with a $k \times k$ filter and no padding turns a side of length n into $n - k + 1$ (you will see why in Chapter 3). A 2×2 pool with stride 2 halves each side. The number of maps after a convolution equals the number of filters.

- 1 Input: $32 \times 32 \times 3$, a colour picture with 3 channels.
- 2 Conv1 with six 5×5 filters: $32 - 5 + 1 = 28$, so $28 \times 28 \times 6$.
six filters give six maps
- 3 Pool1, 2×2 , stride 2: $28 \div 2 = 14$, so $14 \times 14 \times 6$.
pooling never changes the number of maps
- 4 Conv2 with sixteen 5×5 filters: $14 - 5 + 1 = 10$, so $10 \times 10 \times 16$.
- 5 Pool2: $10 \div 2 = 5$, so $5 \times 5 \times 16$.

- 6 Flatten: $5 \times 5 \times 16 = 400$ numbers in one list. FC layer to 120 neurons, then to 10 outputs (one per class). From here on it is Book 5.

This is the shape of LeNet-5, the first famous CNN, which you will meet in Chapter 8.

Σ MATHS, STEP BY STEP

PLAIN MATHS

One conv layer, no padding, filter size k , stride 1, input side n :

$$n_{\text{out}} = n - k + 1$$

One 2×2 pool with stride 2:

$$n_{\text{out}} = n/2$$

Example: $32 \rightarrow 28 \rightarrow 14 \rightarrow 10 \rightarrow 5$.

The general formula, with padding p and stride s , comes in Chapter 4: $\lfloor (n + 2p - k)/s \rfloor + 1$.

→

MATRIX FORM

Every stage is a 3-D box of numbers, a **volume**:

$$\mathbf{X}^{(\ell)} \in \mathbb{R}^{H_\ell \times W_\ell \times C_\ell}$$

H_ℓ, W_ℓ = height and width of the maps at stage ℓ ; C_ℓ = number of maps (channels).

Conv changes all three (C becomes the number of filters); pooling changes only H and W .

Flatten turns the last volume into a vector of length $H \cdot W \cdot C$; the FC layers then work as in Book 5: $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$.

Layer by layer, and the classification process

Your slides 33–34 describe what each part of the network ends up doing. Read this table as "what the network learns to do", not as rules we impose.

Layers	Purpose	Features found	Receptive field
Early (1–2)	Detect low-level features	Edges, corners, lines; small filters (3×3, 5×5)	Local, small regions
Middle (3–4)	Combine low-level features	Shapes, textures, patterns; medium-level abstractions	Larger regions
Deep (5+)	Detect object parts	Wheels, faces, eyes, ears; high-level meaning	Large image regions
Fully connected	Final classification decision	Input: the feature vector from the conv layers; output: class scores	Combines all learned features

Slide 35 lists the classification process as five steps: (1) **feature extraction**, the conv layers pull out features of increasing complexity; (2) **spatial reduction**, pooling shrinks the maps while keeping the important information; (3) **feature combination**, deeper layers combine simple features into complex patterns; (4) **decision making**, the fully connected layers use the features to decide; (5) **output**, softmax turns the scores into probabilities. The key insight on that slide: each layer is a feature detector that becomes more capable than the one before.

? STOP AND THINK

Ravi says: "If every neuron only sees a 3×3 patch, how can the network ever see a whole face? A face is much bigger than 3×3 ."

✓ OUR ANSWER

By stacking. A neuron in layer 2 sees a 3×3 patch of layer 1, and each of those layer-1 neurons saw a 3×3 patch of the picture. So the layer-2 neuron indirectly sees 5×5 pixels. Layer 3 sees 7×7 , and so on. Pooling makes this grow even faster. Deep enough, and a single neuron "sees" the whole face. We work this out with a formula in Chapter 4.

☰ ON YOUR SLIDES

Slide 18: definition and core insight. Slide 19: why CNNs. Slides 20–23: the three properties (bird pictures). Slides 24–27: "The Whole CNN". Slides 28–29: key principles, characteristics. Slides 30–31: hierarchical features, the face example. Slide 32: the pattern with $32 \times 32 \times 3 \dots 10$. Slides 33–35: layer-by-layer breakdown and the classification process.

! CAREFUL

"Parameter sharing" means *one* set of 9 numbers used at every position, not 9 numbers copied to every position. When the network learns, all positions push on the same 9 numbers. That is why a filter that is useful everywhere is what gets learned.

✓ CHECK YOURSELF

1. Name the three properties of images that CNNs use. **Patterns are small (locality); the same pattern appears in different places (translation); sub-sampling keeps the object (scale).**
2. Which principle cuts the number of parameters most? **Parameter sharing: one filter for all positions.**
3. A 28×28 map goes through a 5×5 conv with no padding. New size? $28 - 5 + 1 = 24$.
4. Sixteen filters are applied. How many output maps? **16.**
5. What does the flatten step do? **Turns the last $H \times W \times C$ volume into one list of $H \cdot W \cdot C$ numbers for the FC layers.**

👉 IN ONE BREATH

- Local: a neuron looks at a small patch. Shared: the same filter slides over the whole picture. Shrink: pool between layers.
- Recipe: $\text{INPUT} \rightarrow [\text{CONV} \rightarrow \text{ACT} \rightarrow \text{POOL}] \times N \rightarrow \text{FC} \rightarrow \text{OUTPUT}$.
- Conv: side becomes $n - k + 1$, maps become the number of filters. Pool: side halves, maps unchanged.
- Early layers find edges, middle layers find parts, deep layers find objects, FC layers decide.

Convolution by hand

Slide a small filter over the picture, multiply and add at every spot: a feature map.

On your slides: **Module 6, slides 37–49** · 2-D convolution (cross-correlation) · The 6×6 example, step by step · Feature maps · Edge filters · Multiple feature maps

STORY

Asha wants to see the sliding idea with her own hands before she trusts Nova with it. She draws a tiny 6×6 picture on the whiteboard: ones where the ink is, zeros where the paper is white. Next to it she draws a 3×3 "pattern card" with ones and minus ones. "Hold the card on the top-left corner," she tells Ravi. "Multiply each pair, add them up, write the answer. Now slide the card one square to the right and do it again." After sixteen answers, Nova's screen shows the same sixteen numbers. Two of them are big and positive, and they sit exactly where the diagonal stroke is.

BIG IDEA

Convolution is: put the filter on the picture, multiply each filter number by the picture number underneath it, add all the products, write the sum down, slide the filter one step, repeat. The grid of sums is the **feature map**. A big positive number means "the filter's pattern is here". The filter's numbers are the layer's **weights**, and they are learned.

THINK OF IT LIKE...

A transparent card with a pattern of "+" and "-" marks. You lay the card on a spot of the drawing and score it: every "+" on ink counts +1, every "-" on ink counts -1, marks on white paper count 0. Add the score and write it in a notebook. Slide the card one square right and score again. The card is the *filter*; the spot is the *position*; the score is one entry of the *feature map*; the notebook page, filled in, is the whole map. A high score means "the card's pattern matches here".

The formula

Your slide writes the operation like this. I is the input (the image, or the maps from the layer before). K is the **kernel**, also called the **filter**: a small grid of learnable numbers. The star $\star$ means "convolve".

$$(I \star K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n)$$

Read it slowly. (i, j) is where the top-left corner of the filter sits on the input: row i , column j . (m, n) runs over the filter's own cells, from $(0, 0)$ to $(2, 2)$ for a 3×3 filter. $I(i + m, j + n)$ is the input pixel that lies under filter cell (m, n) . Multiply each pair, add all nine, and that is output cell (i, j) . The output is the feature map, "showing where the pattern was detected".

Strictly, this is **cross-correlation**; true convolution flips the filter first. It makes no difference when the filter is learned, so, as your slide says, "deep learning frameworks implement cross-correlation but call it convolution". We do the same.

The deck's example: a 6×6 picture and two filters

The picture is 6×6 , with two diagonal strokes on the left (top and bottom) and a vertical stroke on the right. There are two filters. Both are 3×3 , so "each filter detects a small pattern (3×3)" — your slide calls this **Property 1**. Filter 1 has +1 down its diagonal and -1 elsewhere: it likes a diagonal stroke. Filter 2 has +1 down its middle column: it likes a vertical stroke. The values inside the filters "are to be learned by the network; we call these parameters or weights". For now we take them as given and just compute.

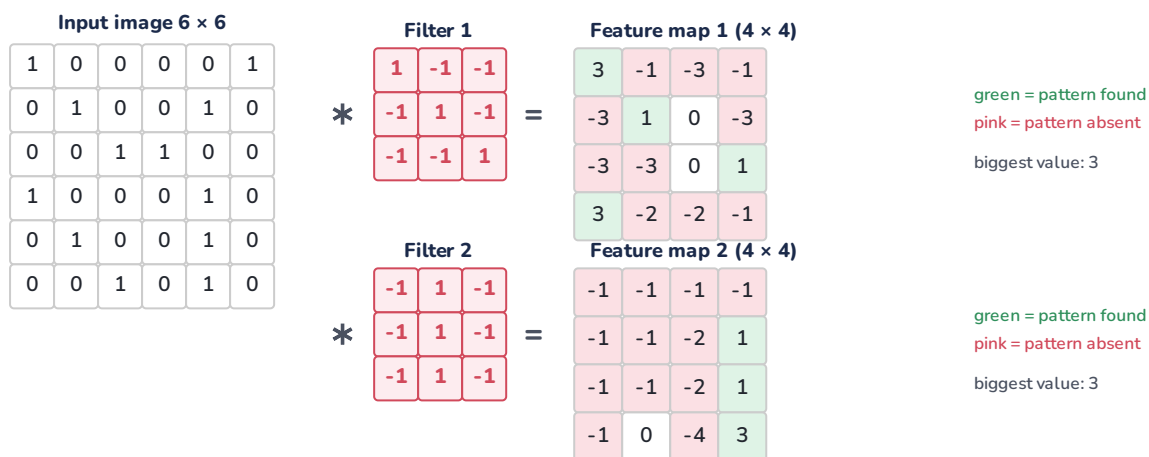


Figure 3.1 The input, the two filters, and the two 4×4 feature maps they produce. Green cells are positive (pattern found), pink cells are negative (pattern absent). Filter 1's map has its 3s where the diagonal strokes are. Filter 2's map has its 3 at the bottom-right, where the vertical stroke is. The rest of this chapter computes these numbers by hand.

WORKED EXAMPLE: FILTER 1, STEP BY STEP (STRIDE 1)

The image I and Filter 1 K are the ones in Figure 3.1. We move the filter with **stride 1**, that is, one pixel at a time.

- 1 **Start at the top-left corner.** The 3×3 window of the image under the filter is $\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$.

rows 0–2, columns 0–2 of I

- 2 **Multiply each pair (window $\times$ filter, cell by cell):** $1 \cdot 1, 0 \cdot (-1), 0 \cdot (-1), 0 \cdot (-1), 1 \cdot 1, 0 \cdot (-1), 0 \cdot (-1), 0 \cdot (-1), 1 \cdot 1$.

- 3 **That is** $1 + 0 + 0 + 0 + 1 + 0 + 0 + 0 + 1 = 3$. **Output cell** $(0, 0) = 3$.
"perform element-wise product and add all the values"

- 4 **Slide one pixel to the right.** New window: columns 1–3, $\begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{pmatrix}$. **Products:**

$0, 0, 0, -1, 0, 0, 0, -1, 1$. **Sum** $= -1$. **Output** $(0, 1) = -1$.

only the three inked cells contribute: two land on -1 , one on $+1$

- 5 **Slide right again: window columns 2–4 gives -3 . Slide right again: columns 3–5 gives -1 .**
The first output row is $3, -1, -3, -1$.
try the columns 2–4 window yourself: the three inked cells all land on -1

- 6 The filter cannot slide further right. At column 4 it would hang over the edge. Your slide: "the filter has to completely superimpose the pixels of the input, NOT partially". So we **jump** to the next row, left-most pixel: rows 1-3, columns 0-2, window $\begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$, products $0, -1, 0, 0, 0, -1, -1, 0, 0$, sum = -3 . Output $(1, 0) = -3$.
- 7 Keep going until the filter has visited every position. A 6×6 image and a 3×3 filter give $6 - 3 + 1 = 4$ positions across and 4 down, so the map is 4×4 .
this is the $n - k + 1$ rule from Chapter 2

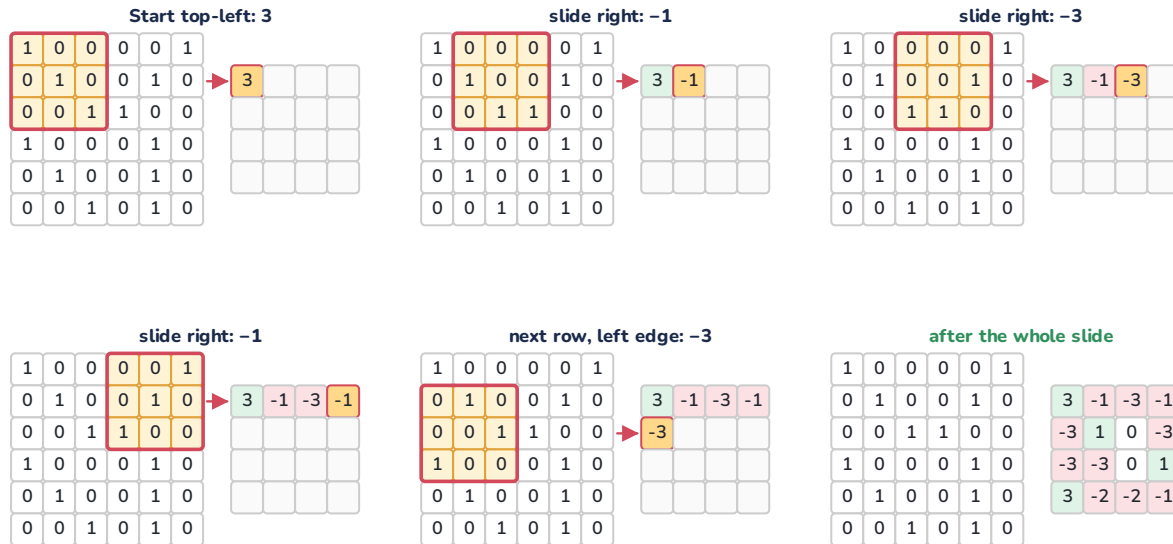


Figure 3.2 The first five positions and the finished map. The red square is the filter's window; the orange cell on the right is the output being written. Follow the top row: 3, -1, -3, -1. Then the jump to the next row: -3. Notice that the window never hangs over the edge, which is why a 6×6 input gives only a 4×4 output.

The finished map for Filter 1 and, by the same method, for Filter 2:

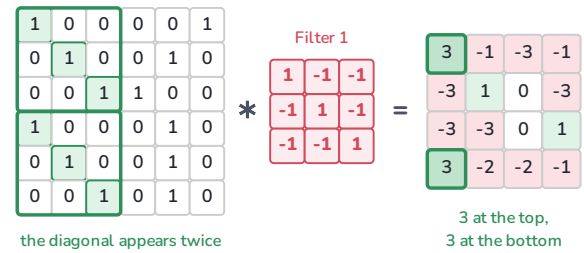
$$I \star K_1 = \begin{pmatrix} 3 & -1 & -3 & -1 \\ -3 & 1 & 0 & -3 \\ -3 & -3 & 0 & 1 \\ 3 & -2 & -2 & -1 \end{pmatrix}, \quad I \star K_2 = \begin{pmatrix} -1 & -1 & -1 & -1 \\ -1 & -1 & -2 & 1 \\ -1 & -1 & -2 & 1 \\ -1 & 0 & -4 & 3 \end{pmatrix}$$

! CAREFUL

Three cells on the slides are printed differently from what the arithmetic gives. Slide 44 prints -1 at row 1, column 1 of Filter 1's map; the window there is $\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{pmatrix}$, products $1, 0, 0, 0, 1, -1, 0, 0, 0$, sum = $+1$. It prints -2 at row 2, column 1; the window is $\begin{pmatrix} 0 & 1 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix}$, products $0, -1, -1, 0, 0, 0, -1, 0, 0$, sum = -3 . And slide 46 prints -1 at row 1, column 3 of Filter 2's map; the window is $\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$, products $0, 1, 0, -1, 0, 0, 0, 1, 0$, sum = $+1$. We print the computed values. Do the three windows yourself; the method matters more than the slip.

Property 2: the same pattern gives the same answer, wherever it is

Look at Filter 1's map again. There is a 3 at the top-left and a 3 at the bottom-left. The picture has the same diagonal stroke in both places. One filter, nine weights, found it twice. Your slide 45 draws the two diagonals and says "Property 2 is satisfied": the same pattern in different places is detected by the same filter with the same parameters. This is parameter sharing doing its job, and it is why the map is equivariant: move the stroke, and the 3 moves with it.



one filter, 9 weights, finds the pattern wherever it sits

Figure 3.3 Two copies of the same diagonal (green cells) and the two 3s they produce. Nine weights served both places.

Σ MATHS, STEP BY STEP

PLAIN MATHS

One output cell, 3×3 filter, filter corner at (i, j) :

$$O(i, j) = \sum_{m=0}^2 \sum_{n=0}^2 I(i+m, j+n) K(m, n)$$

Nine multiplications, eight additions. For $(0, 0)$ above: $O(0, 0) = 1 + 1 + 1 = 3$.

The output has $(n_{\text{in}} - k + 1)$ rows and columns: $6 - 3 + 1 = 4$.

The same nine numbers $K(m, n)$ are used at all 16 positions.

MATRIX FORM

Flatten the filter into a vector of 9: $\mathbf{k} \in \mathbb{R}^9$.

Flatten each of the 16 windows into a row of 9 and stack them: $\mathbf{P} \in \mathbb{R}^{16 \times 9}$ (one row per position).

$$\mathbf{o} = \mathbf{P} \mathbf{k}, \quad \mathbf{o} \in \mathbb{R}^{16}$$

→

$\mathbf{P}$ is 16×9 , $\mathbf{k}$ is 9×1 , so $\mathbf{o}$ is 16×1 : one number per position, reshaped to 4×4 . Every entry of $\mathbf{o}$ is a dot product of a window with the filter — a convolution layer is a neuron with 9 inputs, run at 16 positions with shared weights. This "one matrix multiplication" is how GPUs actually compute it.

What a feature map is

Your slide 47 gives the definition: a **feature map** (or activation map) is the output produced when a filter is applied to an input image or to the previous layer's output. Each filter detects one specific pattern; the map shows *where* that pattern was detected; high values mean strong presence, low values mean weak or absent. With a bias term the slide's formula is:

$$\text{FeatureMap}_k(i, j) = \sum_m \sum_n I(i+m, j+n) \cdot K_k(m, n) + b_k$$

K_k is the k -th filter and b_k is its bias: one extra number per filter, added to every cell of that filter's map. It is the same bias as in every neuron since Book 2. In our example the biases were zero.

EDGE DETECTORS

Slide 48 shows two hand-made filters and what they find. A **vertical-edge** filter has -1 on the left column and $+1$ on the right: it gives a big answer where dark meets bright from left to right. A **horizontal-edge** filter has -1 on top and $+1$ on the bottom. Different filters, different feature maps, "each capturing different aspects of the input".

$$K_{\text{vert}} = \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}, \quad K_{\text{horiz}} = \begin{pmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix}$$

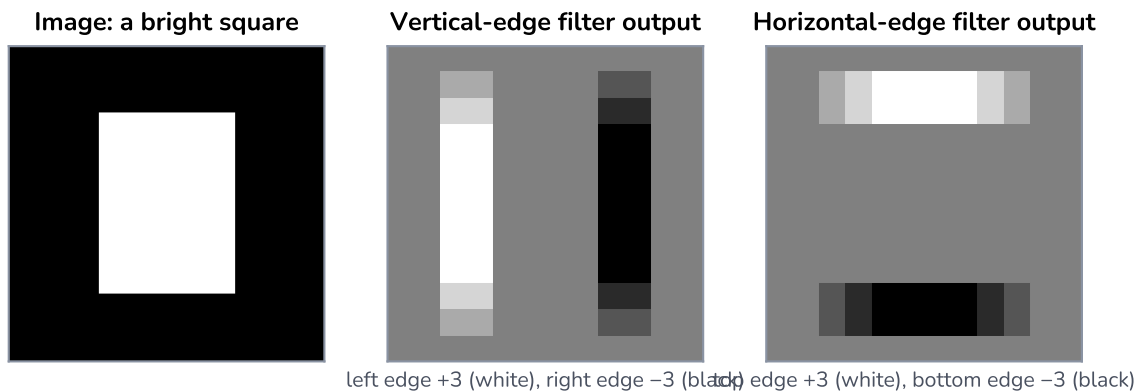


Figure 3.4 A bright square (left) run through the two edge filters. The vertical-edge filter lights up only the left and right sides of the square (and stays grey inside it, where nothing changes). The horizontal-edge filter lights up only the top and bottom. White is a big positive answer, black a big negative one, mid-grey is zero.

Many filters, many maps

One filter finds one pattern. A real layer has many filters, each finding its own pattern and producing its own map. Your slide's example: a $32 \times 32 \times 3$ colour picture and 64 filters of size 3×3 . Each map is $32 - 3 + 1 = 30$ on a side, and there are 64 of them, so the output is $30 \times 30 \times 64$. Three facts from the slide to memorise:

- Number of filters = number of output feature maps.
- Each feature map = one filter's response across the entire input.
- The stack of feature maps is a 3-D volume: height $\times$ width $\times$ channels.

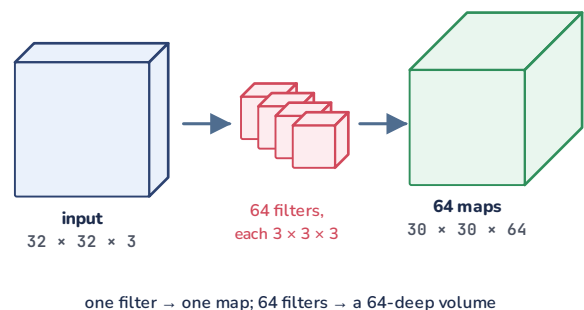


Figure 3.5 Sixty-four filters turn $32 \times 32 \times 3$ into $30 \times 30 \times 64$. The output's depth is the number of filters. (Each filter is really $3 \times 3 \times 3$ because the input has three channels; the third 3 is explained in Chapter 5.)

? STOP AND THINK

Ravi says: "Filter 2's map is -1 almost everywhere. Is it broken?"

✓ OUR ANSWER

No. -1 means "no vertical stroke here". The map has exactly one 3 , at the bottom-right, and that is where the picture's vertical stroke is (column 4, rows 3 to 5). A map full of small numbers with one big one is the perfect answer: the pattern is in one place. In Chapter 5 a ReLU will turn all those negatives into zeros, so only the find remains.

🔍 TRY IT YOURSELF

Compute Filter 2's output at row 3, column 1. The window is rows 3–5, columns 1–3 of the image.

(Window $\begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$; products $0, 0, 0, -1, 0, 0, 0, 1, 0$; sum = 0 , matching the map.)

📄 ON YOUR SLIDES

Slide 37: $(I \star K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n)$, "called convolution". Slide 38: the 6×6 input, Filter 1 and 2, Property 1. Slides 39–44: the sliding steps and the "Output Feature Map". Slide 45: Property 2. Slide 46: both maps. Slide 47: feature map with K_k, b_k . Slide 48: edge detectors. Slide 49: $32 \times 32 \times 3 \rightarrow 30 \times 30 \times 64$.

✓ CHECK YOURSELF

1. In $I(i + m, j + n) \cdot K(m, n)$, what do (i, j) and (m, n) index? (i, j) : the output cell (where the filter's corner sits); (m, n) : the cell inside the filter.
2. Why is the output 4×4 and not 6×6 ? The filter must lie fully inside the image: $6 - 3 + 1 = 4$ positions per side.
3. What does a big positive value in a feature map mean? The filter's pattern is present at that position.
4. A layer has 64 filters. How many feature maps does it output? 64.
5. What does "frameworks call it convolution" refer to? They actually compute cross-correlation (no filter flip), which is fine because the filter is learned.

🔍 IN ONE BREATH

- Put the filter down, multiply pairs, add, write the sum, slide by one. Never hang over the edge.
- $O(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n) + b$. Output side = $n - k + 1$.
- Filter 1 found the diagonal twice (3 and 3); Filter 2 found the vertical stroke once (3). Same filter, any place.
- Many filters $\rightarrow$ many maps $\rightarrow$ a 3-D volume: $32 \times 32 \times 3$ with 64 filters gives $30 \times 30 \times 64$.

How far a neuron can see

Stacked layers see further; padding, stride and dilation control the geometry.

On your slides: **Module 6, slides 50–66** · Receptive field: definition, growth, formula · Learning a kernel · Kernels as parameters · Backprop through convolution · Padding · Stride and the output-size formula · Dilated convolution · Strided vs dilated

📖 STORY

Ravi is still not satisfied. "Your 3×3 card sees three pixels across. My face is a hundred pixels across." Asha draws a second layer on top of the first. "Each cell in the second map is made from a 3×3 patch of the first map. And each cell of the first map came from a 3×3 patch of the picture. So a second-layer cell has seen five pixels across. A third layer sees seven." She keeps counting. Then she frowns at her 6×6 example: "But every layer also makes the map smaller. Six became four. Four will become two. After three layers there is nothing left." She needs a way to stop the shrinking, and a way to shrink on purpose when she wants to.

! BIG IDEA

The patch of the input that one neuron can see is its **receptive field**. It grows by $(k - 1)$ with every extra layer, faster if earlier layers used stride or pooling. Three knobs control the geometry of a convolution: **padding** (add a border of zeros so the output stays the same size), **stride** (jump more than one pixel, so the output shrinks), and **dilation** (spread the filter's cells apart, so it sees a wider patch with the same 9 weights).

≈ THINK OF IT LIKE...

A hostel with a warden, floor monitors and room-mates. A room-mate knows about the 3 rooms around him. A floor monitor talks to 3 room-mates, so she knows about 5 rooms. The warden talks to 3 monitors, so he knows about 7 rooms, though he never visited any of them. Each level "sees" further by listening to the level below. That widening view is the receptive field. Nobody at the top needs to see everything directly.

The receptive field

Your slide's definition: the receptive field of a neuron is the region of the input image that affects that neuron's activation value. Three facts follow. It grows with depth. Early layers have small receptive fields, so they can only find local patterns. Deep layers have large receptive fields, so they can use global context. And it depends on kernel size, stride, and pooling. The trade-off on the slide: a small receptive field cannot capture large objects; a large one understands context better but costs more computation.

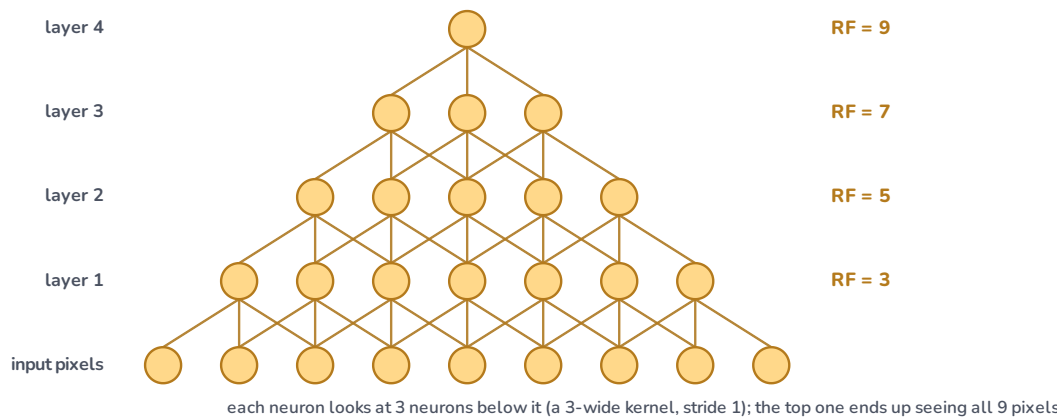


Figure 4.1 A 1-D slice of four stacked 3-wide convolutions with stride 1. Follow the orange connections down from the top neuron: it depends on 3 neurons in layer 3, which depend on 5 in layer 2, 7 in layer 1, and 9 input pixels. So its receptive field is 9 pixels wide, although each single connection only spans 3. In 2-D the same thing happens in both directions: 3×3 , then 5×5 , 7×7 , 9×9 .

Your slide 51 shows a 10×10 input, a layer-1 neuron that "sees 3×3 ", and a layer-2 neuron that "sees 7×7 ", and says the receptive field "approximately doubles with each layer (depends on architecture)". With stride 1 everywhere it grows by 2 per layer, as in Figure 4.1; with pooling or stride between the layers it grows much faster. Slide 52 gives the exact rule:

$$RF_l = RF_{l-1} + (K_l - 1) \prod_{i=1}^{l-1} S_i$$

RF_l is the receptive field at layer l , K_l is that layer's kernel size, and S_i is the stride at each earlier layer i . The product $\prod S_i$ is all the earlier strides multiplied together; with stride 1 everywhere it is just 1.

WORKED EXAMPLE: FOUR 3×3 CONVOLUTIONS, STRIDE 1

- 1 Layer 1: $RF_1 = 3$.
one filter, one 3×3 patch
- 2 Layer 2: $RF_2 = 3 + (3 - 1) \times 1 = 5$.
the product of earlier strides is 1
- 3 Layer 3: $RF_3 = 5 + (3 - 1) \times 1 = 7$.
- 4 Layer 4: $RF_4 = 7 + (3 - 1) \times 1 = 9$.
- 5 Weights used (per channel): four filters of $9 = 36$. One 9×9 filter would need 81 to see the same 9×9 patch.
your slide's key insight: stacking small kernels is more efficient than one large kernel, and it adds a ReLU after each

Where the filter's numbers come from

So far we used filters that someone wrote down. In a CNN nobody writes them. Your slide 53 asks "how does the network learn what values to put in the kernels?" and answers: **gradient descent**. The kernel weights are learnable parameters, exactly like the weights of a fully connected layer. They

start random, they are updated by backpropagation, and the network ends up with kernels that make the loss small. The slide's picture: random kernels → (training) → meaningful edge, texture and pattern detectors.

How many parameters is one filter? Slide 54: for a $K \times K$ filter with C_{in} input channels,

$$\text{parameters per filter} = K \times K \times C_{in} + 1$$

The +1 is the bias. Example on the slide: a 3×3 filter on 64 input channels has $3 \times 3 \times 64 = 576$ weights plus 1 bias, 577 parameters. (The C_{in} is explained in Chapter 5: a filter has one 3×3 slice per input channel.) The update is

$$K_{\text{new}} = K_{\text{old}} - \eta \frac{\partial L}{\partial K}$$

where L is the loss and η (eta) is the learning rate: the same step-downhill rule as always.

WHAT LEARNED KERNELS LOOK LIKE

Slide 55 shows three typical first-layer kernels: a vertical-edge detector, a horizontal-edge detector, and a diagonal-edge detector. First layers learn edge detectors at various orientations and scales. Middle layers learn texture and pattern detectors (blobs, corners). Deep layers learn complex pattern detectors that are hard to draw. The important word on the slide: the network discovers these *automatically* from data.

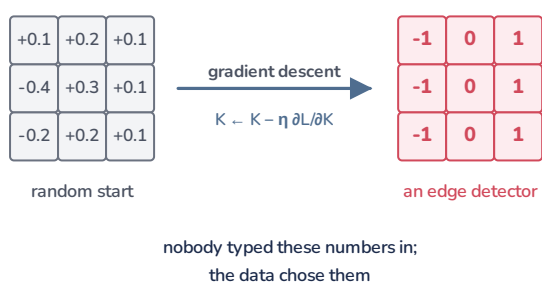


Figure 4.2 A filter starts as nine random numbers. Gradient descent nudges them thousands of times, and they end up as, for example, a vertical-edge detector.

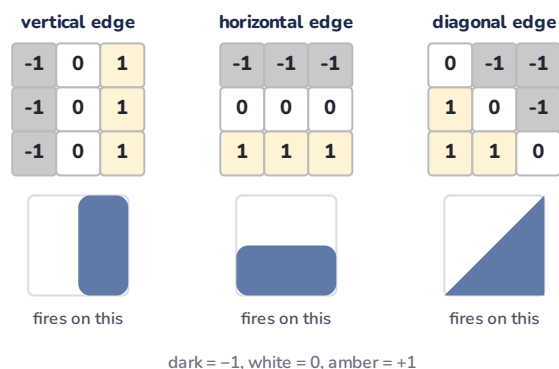


Figure 4.3 The three kernels from your slide and the picture each responds to: "dark on one side, bright on the other" in three directions.

PLAIN MATHS

Forward pass, one output cell:

$$\text{Out}(i, j) = \sum_m \sum_n \text{In}(i + m, j + n) K(m, n) + b$$

Backward pass. Kernel cell $K(m, n)$ touched every output cell, once per position, always multiplied by the input pixel under it. So its gradient adds up over all positions:

$$\frac{\partial L}{\partial K(m, n)} = \sum_i \sum_j \frac{\partial L}{\partial \text{Out}(i, j)} \cdot \text{In}(i + m, j + n)$$

The bias was added to every cell, so:

$$\frac{\partial L}{\partial b} = \sum_i \sum_j \frac{\partial L}{\partial \text{Out}(i, j)}$$

MATRIX FORM

Let $\mathbf{G}$ be the map of $\partial L / \partial \text{Out}(i, j)$, the same size as the output (4×4 in our example). Then the kernel gradient is the input *convolved with* $\mathbf{G}$:

$$\frac{\partial L}{\partial \mathbf{K}} = \text{In} \star \mathbf{G}$$

a 6×6 input and a 4×4 "filter" $\mathbf{G}$ give a 3×3 result: exactly the shape of $\mathbf{K}$. Your slide's key insight: "gradient computation is also a convolution-like operation". The bias gradient is the sum of all entries of $\mathbf{G}$. Book 5's chain rule, nothing new; only the shape is new.

Padding: keep the map from shrinking

Without padding a convolution shrinks the map: output size = input size – kernel size + 1. Slide 57 gives two more reasons to dislike this. Pixels at the border get less attention (a corner pixel is visited by only one window; a middle pixel by nine). And in a deep network the maps would shrink rapidly to nothing. The fix is to add a border of zeros around the input before convolving. That border is **padding**.

- **Valid padding:** no padding; the output is smaller than the input (what we did in Chapter 3).
- **Same padding:** pad just enough that the output has the same size as the input.
- **Full padding:** the maximum padding, so that every partial overlap counts too; the output is bigger than the input.

For same padding with kernel size k and stride 1, the slide gives the amount to add on each side:

$$p = \frac{k - 1}{2}$$

For $k = 3, p = 1$: one ring of zeros. For $k = 5, p = 2$. Slide 58 draws exactly this: the 6×6 image with one ring of zeros becomes 8×8 , and Filter 1 now produces a 6×6 map.

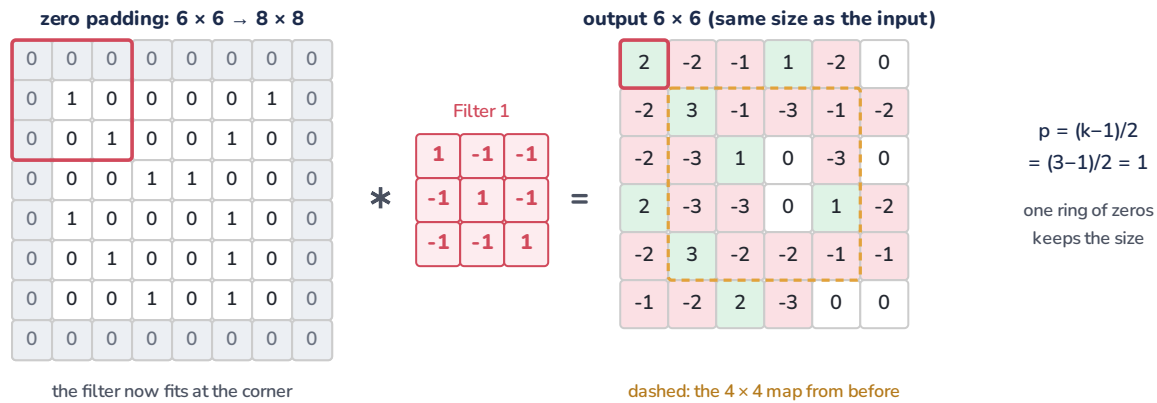


Figure 4.4 The 8×8 zero-padded image from your slide, Filter 1, and the 6×6 output we computed. The corner window now sits on the corner pixel (red). The dashed square is the 4×4 map of Chapter 3, unchanged, sitting in the middle; the outer ring is new information about the borders. Same size in, same size out.

Stride: shrink on purpose

The **stride** s is how far the kernel moves at each step. Stride 1 moves one pixel at a time (standard). Stride 2 moves two pixels at a time and skips every other position, so the output is about half the size: it **downsamples**. Slide 59 gives the general size rule for input size n , kernel k , padding p and stride s :

$$\text{output size} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

The $\lfloor \cdot \rfloor$ means round down to a whole number. The $2p$ is because padding is added on both sides. Check it on Chapter 3: $n = 6, p = 0, k = 3, s = 1$ gives $\lfloor 3/1 \rfloor + 1 = 4$. Correct.

WORKED EXAMPLE: THE SLIDE'S STRIDE-3 CONVOLUTION

Same 6×6 image, same Filter 1, but now the filter jumps 3 pixels at a time.

- 1 Output size: $\lfloor (6 + 0 - 3)/3 \rfloor + 1 = \lfloor 1 \rfloor + 1 = 2$. The map is 2×2 .
- 2 Window at rows 0-2, columns 0-2: the diagonal, sum = 3.
same as Chapter 3's first window
- 3 Jump 3 right: rows 0-2, columns 3-5, window $\begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}$, products 0, 0, -1, 0, 1, 0, -1, 0, 0, sum = -1.
- 4 Jump 3 down, back to the left: rows 3-5, columns 0-2: the second diagonal, sum = 3.
- 5 Rows 3-5, columns 3-5: $\begin{pmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{pmatrix}$, products 0, -1, 0, 0, 1, 0, 0, -1, 0, sum = -1.
- 6 Feature map: $\begin{pmatrix} 3 & -1 \\ 3 & -1 \end{pmatrix}$, as on the slide. Four windows instead of sixteen: four times less work, and a map a quarter of the size.

Slide 62 shows a second example with the pixels numbered 1 to 36, a 3×3 kernel and stride 2. The formula gives $\lfloor (6 - 3)/2 \rfloor + 1 = \lfloor 1.5 \rfloor + 1 = 2$. The floor matters: the windows start at columns 0 and 2; a window starting at column 4 would need columns 4-6, and there is no column 6, so it is

dropped. The slide's trade-off: lose spatial resolution, gain computational efficiency.

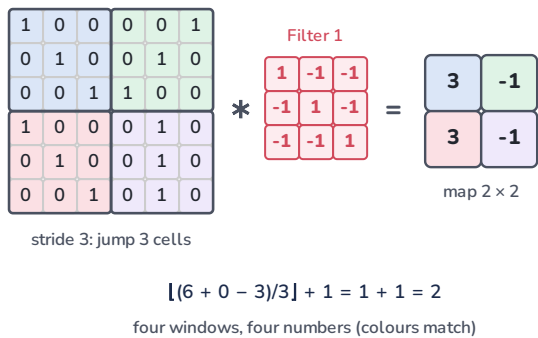


Figure 4.5 Stride 3 cuts the image into four non-overlapping blocks (four colours); each block gives one output cell of the same colour.

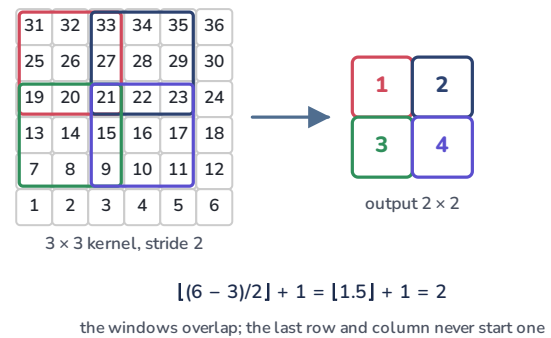


Figure 4.6 The slide's stride-2 example: four overlapping windows (coloured frames) give a 2 × 2 output; the last row and column never start a window.

Slide 61 lists why you would use a strided convolution: spatial downsampling (smaller maps), replacing pooling (learn the downsampling instead of a fixed max), computational efficiency (fewer computations in later layers), and a bigger receptive field per neuron.

Dilation: see wider with the same nine weights

A **dilated** (or **atrous**) convolution inserts gaps between the kernel's cells. The **dilation rate** r is the spacing: $r = 1$ is an ordinary convolution, $r = 2$ leaves one pixel gap between kernel cells, $r = 3$ leaves two. The kernel still has 9 weights, but it now covers a wider patch. Slide 63's formula for the effective size:

$$k_{\text{eff}} = k + (k - 1)(r - 1)$$

For $k = 3$ and $r = 2$: $k_{\text{eff}} = 3 + 2 \times 1 = 5$. A 3 × 3 kernel with dilation 2 has a 5 × 5 receptive field with 9 parameters. Your slide: "same number of parameters (9), but larger receptive field!"

Slide 65 says when to use it: semantic segmentation (a large receptive field at full resolution), dense prediction (detection, depth estimation), and audio (WaveNet). Benefits: larger receptive field without pooling or striding; resolution preserved; same parameter count; multi-scale context by mixing rates. Trade-offs: it may lose fine local detail; "gridding" artefacts can appear (reduced by varying the rate); slightly more memory in the forward pass.

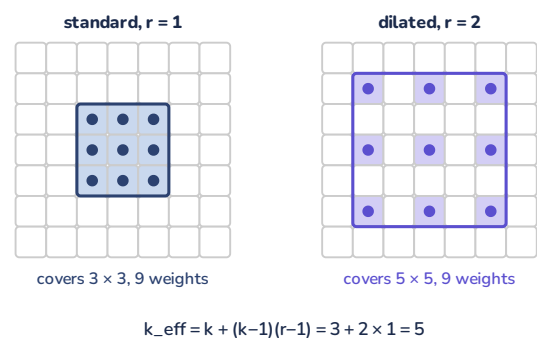


Figure 4.7 The same nine weights: packed (3 × 3) or spread out with dilation rate 2 (5 × 5). Nothing was added; the dots were spread.

Property	Strided	Dilated
Output size	Reduced	Same as input
Receptive field	Increased	Increased
Parameters	Same	Same
Downsampling	Yes	No
Resolution loss	Yes	No
Typical use	Classification	Segmentation

Slide 66's summary: strided convolution reduces spatial size (classification); dilated convolution increases the receptive field while keeping resolution (dense prediction); both are alternatives to pooling, for different purposes.

🔍 STOP AND THINK

Ravi says: "If you want a 9×9 view, why not just use a 9×9 filter? Why stack four small ones?"

✅ OUR ANSWER

Count the weights. One 9×9 filter has 81 per channel. Four 3×3 filters have $4 \times 9 = 36$. Same view, less than half the weights. And between the four small filters you get three extra ReLUs, so the stack can learn more complicated patterns than one big linear filter. This is the idea behind VGG in Chapter 8.

📄 ON YOUR SLIDES

Slides 50–52: receptive field, the 10×10 visual, $RF_l = RF_{l-1} + (K_l - 1) \prod S_i$, 3, 5, 7, 9. Slides 53–56: learning a kernel, $K \times K \times C_{in} + 1 = 577$, the update, learned kernels, backprop formulas. Slides 57–58: padding, $(k - 1)/2$, the 8×8 picture. Slides 59–62: stride, $\lfloor (n + 2p - k)/s \rfloor + 1$, the stride-3 map (3, -1 / 3, -1), the numbered stride-2 grid. Slides 63–66: dilation, k_{eff} , uses, the comparison table.

⚠️ CAREFUL

In the size formula, p is the padding on *one* side, so the formula uses $2p$. And always take the floor: $\lfloor 1.5 \rfloor = 1$, not 2. A window that would hang over the edge simply does not exist.

✅ CHECK YOURSELF

1. Three stacked 3×3 convolutions, stride 1. Receptive field? **7×7 (3 → 5 → 7).**
2. How many parameters in a 5×5 filter on 3 input channels? **$5 \times 5 \times 3 + 1 = 76$.**
3. Same padding for a 5×5 kernel: how many zeros on each side? **$p = (5 - 1)/2 = 2$.**
4. Input 32, kernel 5, padding 0, stride 2. Output size? **$\lfloor 27/2 \rfloor + 1 = 13 + 1 = 14$.**
5. A 3×3 kernel with dilation rate 3: effective size and parameter count? **$3 + 2 \times 2 = 7$, so 7×7 ; still 9 parameters.**

🔴 IN ONE BREATH

- Receptive field = the input patch one neuron can see. It grows by $k - 1$ per layer (times the earlier strides): 3, 5, 7, 9.
- Kernels are learned by gradient descent; $K \times K \times C_{in} + 1$ parameters each; the kernel gradient is itself a convolution.
- Padding $p = (k - 1)/2$ keeps the size. Stride s shrinks it: $\lfloor (n + 2p - k)/s \rfloor + 1$.
- Dilation spreads the kernel: $k_{eff} = k + (k - 1)(r - 1)$, same 9 weights, wider view.

Depth, shrinking, and switching on

Filters have depth, pooling shrinks for free, 1×1 mixes channels, ReLU switches off misses.

On your slides: **Module 6, slides 67–84** · Multi-channel input and output · The $4 \times 4 \times 2$ example · Channel evolution · Pooling: max and average · 1×1 convolution and the bottleneck · ReLU · Conv + ReLU + max pool

STORY

Asha's camera is a colour camera: every picture arrives as three grids, red, green, blue. "So which grid does the card slide over?" asks Ravi. "All three," says Asha. "The card gets three layers, one per colour, and I add the three scores." Then she looks at her plan for Nova: 64 filters in the first layer, 128 in the second. "That is a lot of maps, and Nova's memory is small." She needs to shrink the maps as she goes, and to throw away the -1s that mean "nothing here".

BIG IDEA

A filter has one slice per input channel, so its depth equals the input's depth; the slices' answers are added, plus one bias, to give one map. Many filters give many maps. Between layers, **pooling** halves the height and width with no parameters at all. A 1×1 convolution mixes channels at one pixel and is the cheap way to cut channel counts. **ReLU** after every convolution turns "pattern absent" (negative) into exactly 0.

Filters for inputs with channels

Real inputs have channels: RGB has 3, grayscale 1, and the maps inside a CNN typically 64, 128, 256 or 512. Slide 67 asks: how does convolution work on a multi-channel input, how do we get a multi-channel output, and how many parameters do we need?

THINK OF IT LIKE...

Three stencils glued into a small book, one page per colour sheet. You lay the book on a spot: the red page scores the red sheet, the green page scores the green sheet, the blue page scores the blue sheet. Add the three scores, add the bias, write one number. One book (one filter, three pages deep) gives one map. A second book gives a second map.

Slide 68's key point: **the filter has the same depth as the input**. On an $H \times W \times 3$ image, a filter is $k \times k \times 3$, and it produces a single map of size $H' \times W' \times 1$. The formula adds a third sum, over channels c :

$$\text{Output}(i, j) = \sum_{c=1}^3 \sum_m \sum_n I(i+m, j+n, c) \cdot K(m, n, c) + b$$

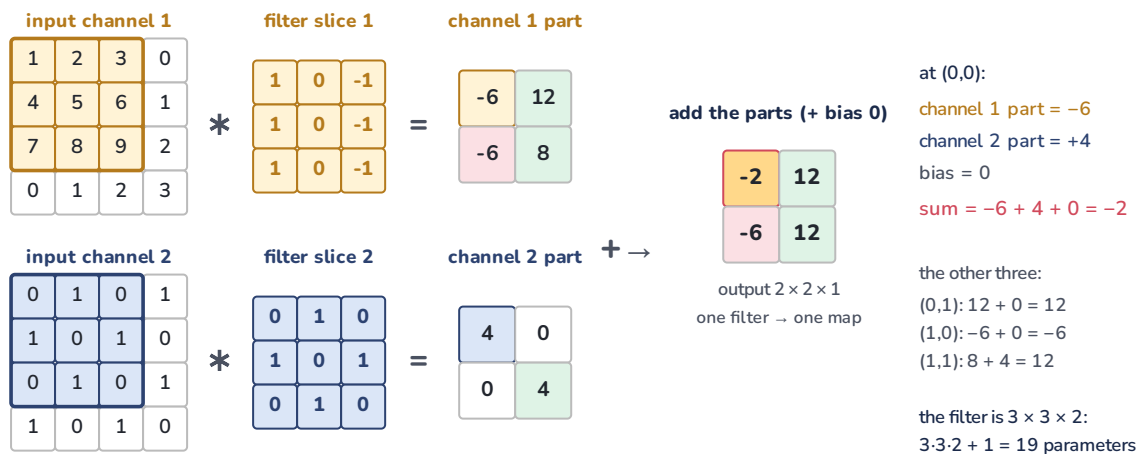
Slide 69 splits it into two steps. Step 1: for each channel c , do an ordinary 2-D convolution with that channel's slice of the filter, $O_c(i, j) = \sum_m \sum_n I(i+m, j+n, c) K(m, n, c)$. Step 2: add the per-channel results and the bias, $O(i, j) = \sum_c O_c(i, j) + b$. One filter produces one output channel. Parameters per filter: $k \times k \times C_{in} + 1$; the slide's example is $3 \times 3 \times 64 + 1 = 577$ again.

WORKED EXAMPLE: THE SLIDE'S $4 \times 4 \times 2$ INPUT (ALL FOUR OUTPUTS)

Input: two channels, each 4×4 . Filter: $3 \times 3 \times 2$, one slice per channel. Stride 1, no padding, bias 0, so the output is $4 - 3 + 1 = 2$ on each side: $2 \times 2 \times 1$. The slide computes the sum for channel 1 at position (0,0) and stops; we do all four positions.

$$I_1 = \begin{pmatrix} 1 & 2 & 3 & 0 \\ 4 & 5 & 6 & 1 \\ 7 & 8 & 9 & 2 \\ 0 & 1 & 2 & 3 \end{pmatrix}, I_2 = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}, K_1 = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}, K_2 = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

- 1 Position (0,0), channel 1: $1(1) + 2(0) + 3(-1) + 4(1) + 5(0) + 6(-1) + 7(1) + 8(0) + 9(-1) = 1 - 3 + 4 - 6 + 7 - 9 = -6$.
 K_1 is a vertical-edge filter: left column minus right column
- 2 Position (0,0), channel 2: $0(0) + 1(1) + 0(0) + 1(1) + 0(0) + 1(1) + 0(0) + 1(1) + 0(0) = 4$.
 K_2 's four 1s land exactly on four 1s of the checkerboard
- 3 Add the parts and the bias: $-6 + 4 + 0 = -2$. Output (0,0) = -2.
- 4 Position (0,1): channel 1 window is columns 1-3 of I_1 : $2 + 5 + 8 - (0 + 1 + 2) = 15 - 3 = 12$; channel 2 window has its 1s on K_2 's zeros, giving 0. Output = $12 + 0 = 12$.
- 5 Position (1,0): channel 1, rows 1-3: $4 + 7 + 0 - (6 + 9 + 2) = 11 - 17 = -6$; channel 2: 0. Output = -6.
- 6 Position (1,1): channel 1: $5 + 8 + 1 - (1 + 2 + 3) = 14 - 6 = 8$; channel 2: 4. Output = $8 + 4 = 12$.
- 7 The output map is $\begin{pmatrix} -2 & 12 \\ -6 & 12 \end{pmatrix}$. This one filter has $3 \times 3 \times 2 + 1 = 19$ parameters.



one filter has one slice per input channel; each slice works on its own channel, then the parts are added

Figure 5.1 Each channel is convolved with its own slice of the filter, then the two partial maps are added (plus the bias). Notice the filter has exactly as many slices as the input has channels, and that the output is one map, not two.

MANY FILTERS: MANY OUTPUT CHANNELS

Slide 70: to get C_{out} output channels, use C_{out} filters, each of size $k \times k \times C_{in}$. The general rule:

PLAIN MATHS

One filter, one output cell:

$$O(i, j) = \sum_{c=1}^{C_{in}} \sum_m \sum_n I(i+m, j+n, c) K(m, n, c) + b$$

Its parameters: $k \cdot k \cdot C_{in}$ weights + 1 bias.

Filter f gives map f ; C_{out} filters give C_{out} maps.

Total parameters of the layer:

$$(k \times k \times C_{in} + 1) \times C_{out}$$

VOLUMES

Input: $H \times W \times C_{in}$. Filters: C_{out} of them, each $k \times k \times C_{in}$. Output: $H' \times W' \times C_{out}$.

Example: 128 filters of 3×3 on 64 channels: $(3 \cdot 3 \cdot 64 + 1) \times 128 = 577 \times 128 = 73,856$ parameters, whatever H and W are.

Slide 73's takeaways: filter depth = input depth; number of filters = number of output channels; each filter works on all input channels at once.

Channels grow as the maps shrink

Slide 72 shows the typical shape of a deep CNN: $224 \times 224 \times 3 \rightarrow 112 \times 112 \times 64 \rightarrow 56 \times 56 \times 128 \rightarrow 28 \times 28 \times 256 \rightarrow 14 \times 14 \times 512$. Each stage halves the height and width and doubles the channels: high resolution with few channels early (local patterns), low resolution with many channels deep (semantic features). Halving H and W divides the work by 4 and doubling C_{in} and C_{out} multiplies it by 4, so the work per layer stays about the same; the slide writes " $H \times W \times C$ approximately constant". Slide 73: **memory** $\propto H \times W \times C$; deep networks trade spatial resolution for channel depth.

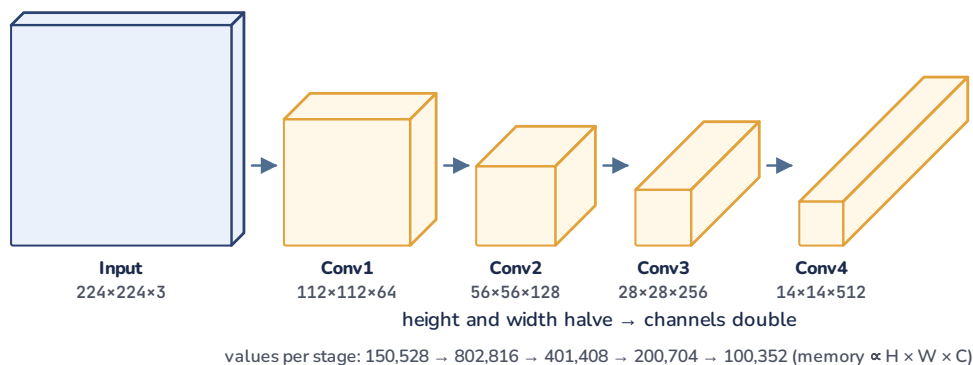


Figure 5.2 The volumes get thinner in height and width, and longer in channels. The number of values does not explode: it halves each stage (150K → 803K → 401K → 201K → 100K), and the multiply-adds per layer stay level.

Pooling

Slide 74: pooling does **spatial downsampling** (reduce height and width, keep the depth), gives **translation invariance** (less sensitive to small shifts), **computational efficiency** (fewer computations later), and **overfitting reduction** (a kind of regularisation). Pooling layers are **parameter-free** but still important. **Global average pooling** (GAP), which averages a whole map to one number, replaces fully connected layers (Chapter 8). Two types: max and average.

WORKED EXAMPLE: MAX AND AVERAGE POOLING (K = 2, S = 2)

Input 4×4 : $\begin{pmatrix} 2 & 4 & 7 & 4 \\ 6 & 6 & 9 & 8 \\ 3 & 4 & 8 & 3 \\ 7 & 5 & 3 & 6 \end{pmatrix}$. Windows are 2×2

and jump by 2, so they do not overlap: four windows, output 2×2 .

- 1 Top-left window $\{2, 4, 6, 6\}$: $\max = 6$; $\text{average} = (2 + 4 + 6 + 6)/4 = 18/4 = 4.5$.
- 2 Top-right $\{7, 4, 9, 8\}$: $\max = 9$; $\text{average} = 28/4 = 7$.
- 3 Bottom-left $\{3, 4, 7, 5\}$: $\max = 7$; $\text{average} = 19/4 = 4.75$.
- 4 Bottom-right $\{8, 3, 3, 6\}$: $\max = 8$; $\text{average} = 20/4 = 5$.
- 5 Max pooling gives $\begin{pmatrix} 6 & 9 \\ 7 & 8 \end{pmatrix}$; average pooling gives $\begin{pmatrix} 4.5 & 7 \\ 4.75 & 5 \end{pmatrix}$. Both match the slides.

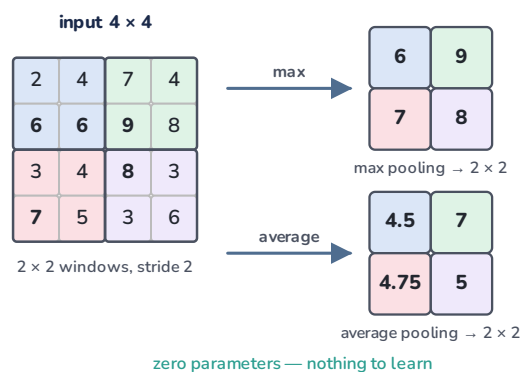


Figure 5.3 The four windows and the two results. Max keeps the strongest activation (slide 75: the most common); average is smoother, sometimes used in final layers (slide 76).

The 1×1 convolution

Slide 77 defines it: a 1×1 convolution uses a kernel of size $1 \times 1 \times C_{in}$ to take a weighted sum *across channels* at each pixel. At first it seems pointless: no neighbours, just one pixel. But it is very powerful; the idea comes from the "Network in Network" paper (Chapter 8). It combines information across channels, acts as a learned linear combination, does not look at neighbouring pixels, and is applied at every position independently. Slide 78's formula, at pixel (i, j) :

$$\text{Output}(i, j) = \sum_{c=1}^{C_{in}} \text{Input}(i, j, c) \cdot w_c + b$$

Slide 79's three uses. **Channel reduction**: 64 filters of $1 \times 1 \times 256$ turn $28 \times 28 \times 256$ into $28 \times 28 \times 64$, keeping the spatial size. **Non-linearity**: 1×1 conv then ReLU adds a non-linear step almost for free. **Efficiency**: reduce channels before an expensive 3×3 or 5×5 convolution, as Inception and ResNet do.

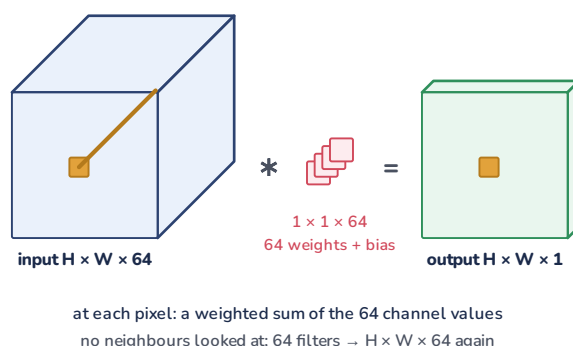


Figure 5.4 A 1×1 filter is a pin of 64 weights through the depth at one pixel. Nothing spatial happens; only channels get mixed.

WORKED EXAMPLE: THE BOTTLENECK (SLIDE 80)

Compare a 5×5 convolution from 256 channels to 256 channels, with and without 1×1 convolutions around it (biases ignored).

- 1 Without: one 5×5 layer, $5 \times 5 \times 256 \times 256 = 1,638,400$ weights.
- 2 With: 1×1 from 256 to 64: $256 \times 64 = 16,384$. Then 5×5 from 64 to 64: $5^2 \times 64 \times 64 = 25 \times 4096 = 102,400$. Then 1×1 from 64 back to 256: $64 \times 256 = 16,384$.
- 3 Total with: $16,384 + 102,400 + 16,384 = 135,168$. Ratio: $1,638,400 / 135,168 \approx 12$.

This "squeeze, work, expand" shape is the **bottleneck** design pattern (Chapters 7 and 8).

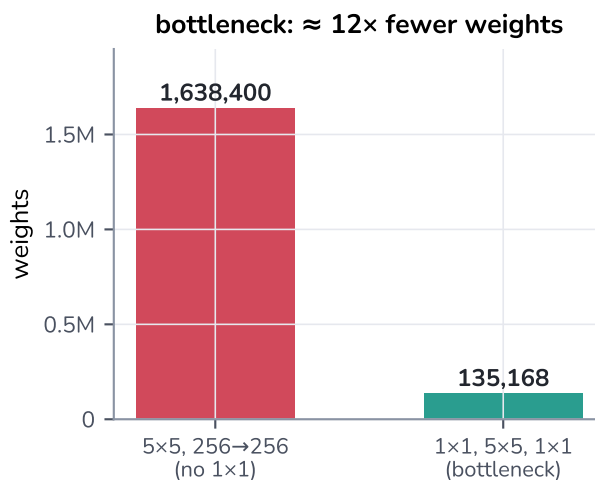


Figure 5.5 The same job, 256 channels in and out, with and without the 1×1 squeeze: a twelfth of the weights.

! CAREFUL

Slide 80 writes the same sum $256 \times 64 + 5^2 \times 64^2 + 64 \times 256$ but prints the total as 147,456 and "11× reduction". The three terms add to 135,168 (about 12×). The formula on the slide is right; only the printed total slipped. Either way the message is the same: about a tenth of the weights.

ReLU in CNNs

You met ReLU in Book 5. Slide 81: it outputs the input if positive, otherwise zero. Properties: non-linear (so stacked layers do not collapse into one), **sparse** (many outputs are exactly zero), computationally cheap (one comparison), and non-saturating (no upper bound on the positive side). In CNNs it is the default activation, applied after each convolution: *conv output* → *ReLU* → *next layer*.

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases}, \quad \frac{d}{dx} \text{ReLU}(x) = \begin{cases} 1 & x > 0 \\ 0 & x < 0 \\ \text{undefined} & x = 0 \end{cases}$$

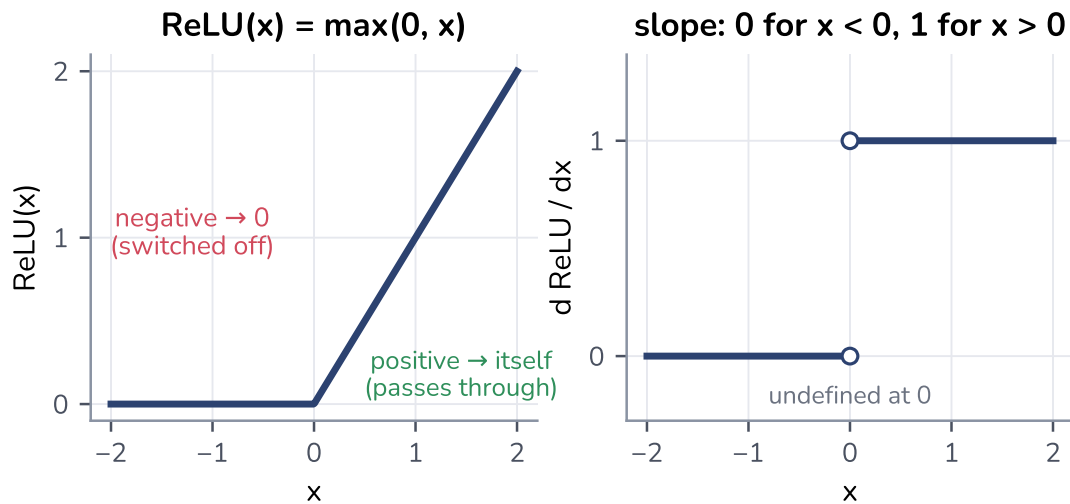


Figure 5.6 The function and its slope. Negative inputs are switched off (output 0, slope 0); positive inputs pass through unchanged (slope 1). At exactly 0 the slope is undefined; code simply picks 0 or 1.

Slide 83, advantages: no vanishing gradient on the positive side (slope 1, not a fraction); cheap (no exp or log); sparse activations; biologically plausible; **scale invariant**, $\text{ReLU}(\alpha x) = \alpha \text{ReLU}(x)$ for $\alpha > 0$. Disadvantages: the **dying ReLU** problem (a neuron whose input is always negative outputs 0 forever and stops learning); outputs not zero-centred (always ≥ 0); not differentiable at zero.

All three together on the 6×6 picture

Slide 84 chains the three operations on Chapter 3's picture with Filter 2 (the vertical-stroke detector): convolve with stride 1, apply ReLU, then max-pool with $k = 2, s = 2$.

WORKED EXAMPLE: CONV → RELU → MAX POOL

1 Convolution (from Chapter 3):
$$\begin{pmatrix} -1 & -1 & -1 & -1 \\ -1 & -1 & -2 & 1 \\ -1 & -1 & -2 & 1 \\ -1 & 0 & -4 & 3 \end{pmatrix}.$$

2 ReLU: every negative becomes 0, positives stay:
$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 3 \end{pmatrix}.$$

"no stroke here" is now exactly 0

3 Max pool, 2×2 windows, stride 2: top-left $\{0, 0, 0, 0\} \rightarrow 0$; top-right $\{0, 0, 0, 1\} \rightarrow 1$; bottom-left $\{0, 0, 0, 0\} \rightarrow 0$; bottom-right $\{0, 1, 0, 3\} \rightarrow 3$.

4 Result: $\begin{pmatrix} 0 & 1 \\ 0 & 3 \end{pmatrix}$. Thirty-six numbers became four, and the four say "vertical stroke on the right, strongest at the bottom".

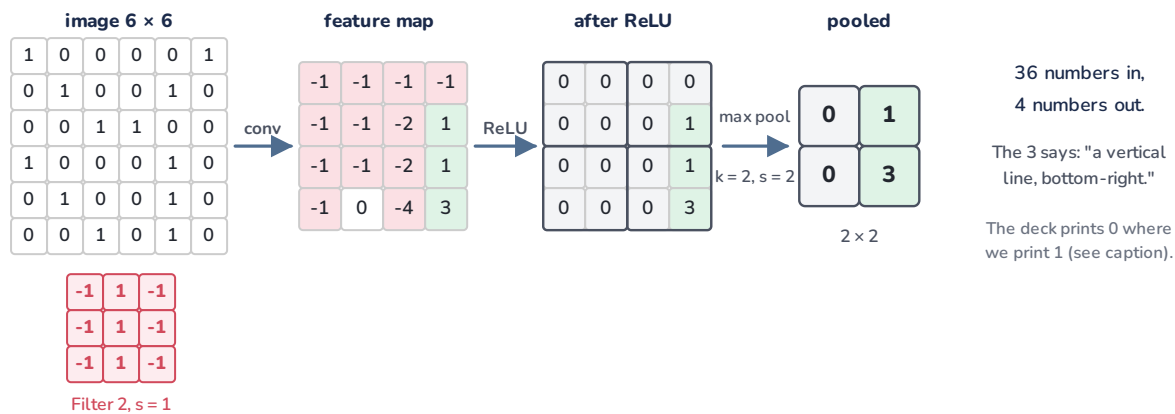


Figure 5.7 The whole chain: the map's strong find survives the ReLU and the pooling; everything else collapses to 0. Detect, switch off the misses, shrink. (Slide 84 prints 0, 0/0, 3 because of the slip at row 1, column 3 noted in Chapter 3; with the correct +1 there the top-right window gives 1.)

? STOP AND THINK

Ravi says: "Max pooling throws away three of every four numbers. How is that not a disaster?"

✓ OUR ANSWER

Because a later layer asks "was the pattern found around here?", not "exactly which pixel?". The maximum answers that, and answers the same even if the stroke moves one pixel inside the window: that is the small-shift invariance. The price is a little lost position, which we wanted to lose anyway.

📄 ON YOUR SLIDES

Slides 67–70: multi-channel formulas, " $(k \times k \times C_{in} + 1) \times C_{out}$ ". Slide 71: the $4 \times 4 \times 2$ example (it stops after "Sum from Ch1 ="; the answer is -6 , output -2). Slides 72–73: channel evolution, memory. Slides 74–76: pooling. Slides 77–80: 1×1 , bottleneck. Slides 81–83: ReLU. Slide 84: the chain.

✓ CHECK YOURSELF

1. Input $56 \times 56 \times 128$, 256 filters of 3×3 . Filter depth, and layer parameters? **128; $1153 \times 256 = 295,168$.**
2. Max pool 2×2 , stride 2 on $56 \times 56 \times 128$: output, parameters? **$28 \times 28 \times 128$; zero.**
3. What does a 1×1 convolution look at? **All channels of one pixel; no neighbours.**
4. Why do channels grow while maps shrink? **Deep layers need many features at low resolution; work per layer stays constant.**

🔥 IN ONE BREATH

- Filter depth = input depth: convolve each channel with its slice, add, add the bias. Layer parameters $(k \cdot k \cdot C_{in} + 1) C_{out}$.
- Maps shrink, channels grow ($224 \times 224 \times 3 \rightarrow 14 \times 14 \times 512$). Pooling: max or mean of each window, no parameters. 1×1 conv mixes channels; the bottleneck cuts weights $\approx 12\times$.
- ReLU zeroes the misses. $6 \times 6 \rightarrow \text{conv (Filter 2)} \rightarrow \text{ReLU} \rightarrow \text{max pool} = 0, 1/0, 3$.

The head that decides

Flatten, score with a fully connected layer, turn scores into probabilities with softmax.

On your slides: **Module 6, slides 85–91** · Fully connected (dense) layer: definition, role, $y = f(Wx + b)$, parameter count, numerical example, pros and cons, alternatives · Softmax: definition, derivative, simplex, numerical example

STORY

Nova's conv layers now produce a small stack of maps: 5×5 , sixteen of them. "So which of these says it is me?" asks Asha. None of them alone. The maps say things like "eye-shaped thing here", "curve there". Something has to weigh all 400 numbers together and give three scores: Asha, cat, neither. Asha smiles. "That is a Book 5 network. I already know how to build that." She flattens the maps into one list and wires up a fully connected layer.

BIG IDEA

The **fully connected** (FC, or dense) layer is Book 5's layer: every input connects to every output, $y = f(Wx + b)$. In a CNN it sits at the end. **Flatten** turns the $H \times W \times C$ maps into a vector of $H \cdot W \cdot C$ numbers; the FC layers map that vector to K class scores; **softmax** turns the scores into probabilities that add up to 1.

THINK OF IT LIKE...

A panel of judges at the end of a talent show. Every judge (output neuron) listens to every act's every detail (every input number) and gives each detail its own weight. Each judge writes down one score. Then the scores are turned into "shares of the prize" that add up to 100%: the biggest score gets the biggest share, but nobody gets exactly zero. The judges are the FC layer; the shares are softmax.

The fully connected layer

Slide 85's definition: a layer where every neuron is connected to every neuron in the previous layer, doing a linear transformation followed by an activation. Its roles in a CNN: **feature flattening**, $(H \times W \times C) \rightarrow (H \cdot W \cdot C)$; the **classification head** that maps features to class scores; **dimensionality control** (choose the size of the feature vector); and **non-linear mapping** (learn complex decision boundaries). The typical flow: conv layers $\rightarrow$ flatten $\rightarrow$ FC layer(s) $\rightarrow$ output.

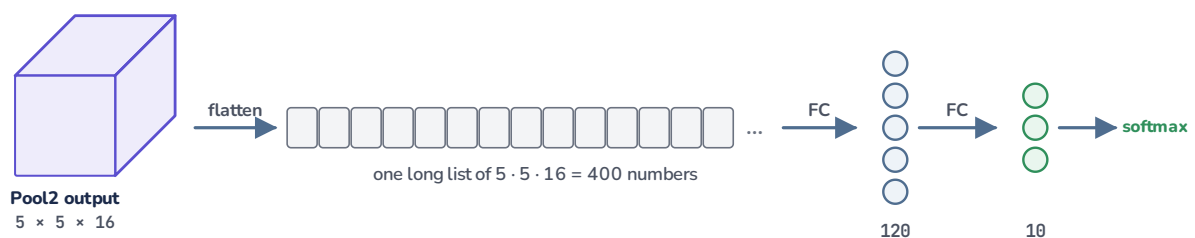


Figure 6.1 The example of Chapter 2, finished. The $5 \times 5 \times 16$ volume is unrolled into 400 numbers, an FC layer maps them to 120, another to 10 scores, and softmax makes probabilities. From flatten onward everything is Book 5.

Σ MATHS, STEP BY STEP

PLAIN MATHS

One output neuron j with n inputs:

$$z_j = \sum_{i=1}^n w_{ji}x_i + b_j, \quad y_j = f(z_j)$$

w_{ji} is the weight from input i to output j ; b_j is the neuron's bias; f is the activation (ReLU inside, softmax at the very end).

Parameters for m outputs: $n \times m$ weights and m biases,

$$n \times m + m = m(n + 1)$$

MATRIX FORM

$$\mathbf{y} = f(\mathbf{W}\mathbf{x} + \mathbf{b})$$

$\mathbf{x} \in \mathbb{R}^n$ input vector, $\mathbf{W} \in \mathbb{R}^{m \times n}$ weight matrix, $\mathbf{b} \in \mathbb{R}^m$ bias vector, $\mathbf{y} \in \mathbb{R}^m$ output. $\mathbf{W}\mathbf{x}$ is $(m \times n)(n \times 1) = m \times 1$: one score per output neuron.

This is the deck's slide 86 exactly, drawn as $n = 4$ inputs and $m = 3$ outputs with w_{11} and w_{31} labelled.

WORKED EXAMPLE: THE SLIDE'S FC LAYER (SLIDE 87)

Input $\mathbf{x} = (2, -1, 3)$; two output neurons with $\mathbf{W} = \begin{pmatrix} 0.5 & -0.2 & 0.1 \\ -0.3 & 0.4 & 0.2 \end{pmatrix}$ and $\mathbf{b} = (0.1, -0.2)$; activation ReLU.

- 1 $z_1 = 0.5(2) + (-0.2)(-1) + 0.1(3) + 0.1 = 1.0 + 0.2 + 0.3 + 0.1 = 1.6$.
first row of $\mathbf{W}$ dotted with $\mathbf{x}$, plus b_1
- 2 $z_2 = (-0.3)(2) + 0.4(-1) + 0.2(3) + (-0.2) = -0.6 - 0.4 + 0.6 - 0.2 = -0.6$.
- 3 $y_1 = \text{ReLU}(1.6) = 1.6$; $y_2 = \text{ReLU}(-0.6) = 0.0$. Output $\mathbf{y} = (1.6, 0.0)$.
- 4 Parameters: $3 \times 2 + 2 = 8$.
 $m(n + 1) = 2 \times 4$

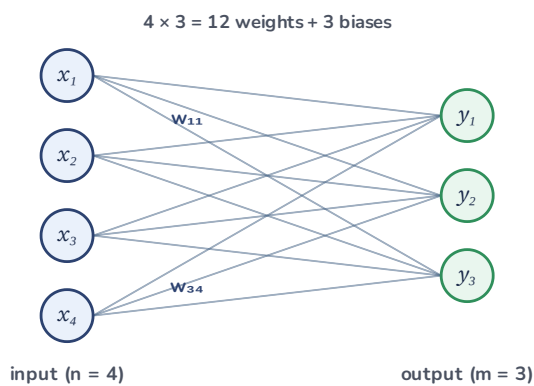


Figure 6.2 The slide's FC picture: 4 inputs, 3 outputs, every pair connected; $3(4 + 1) = 15$ parameters.

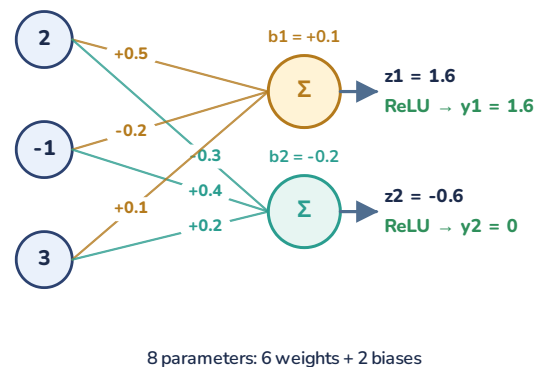


Figure 6.3 The worked example as a picture: amber edges feed neuron 1, teal edges neuron 2, weights on the edges.

FOR AND AGAINST, AND THE MODERN ALTERNATIVES

Slide 88. Advantages: universal approximation (can learn any function), flexible output size, non-linear transformations, and it integrates all spatial information. Disadvantages: **parameter heavy** (overfitting risk; VGG-16's first FC layer alone has over 100 million weights), spatial information is

lost, the input size is fixed, and the matrix multiplication is costly. Modern alternatives: **global average pooling** (average each map to one number, no parameters) and **1 × 1 convolutions** (keep the spatial structure while reducing channels). Both appear in Chapter 8.

Softmax: scores into probabilities

Slide 89: softmax converts a vector of real numbers into a probability distribution, and is used in the output layer for multi-class classification: *final conv/FC layer* → *softmax* → *class probabilities*. You met it in Book 4; here is the deck's version. For $\mathbf{z} = (z_1, \dots, z_K)$ with K classes,

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad \sigma(\mathbf{z}) = (\sigma_1, \dots, \sigma_K)$$

Every σ_i is between 0 and 1 and they add to 1. The slide draws the outputs for $K = 3$ as a triangle (the **2-simplex**): the corners are the certain answers $(1, 0, 0)$, $(0, 1, 0)$, $(0, 0, 1)$, and every valid output is a point inside. Its derivative has two cases, needed for backpropagation:

$$\frac{\partial \sigma_i}{\partial z_j} = \begin{cases} \sigma_i(1 - \sigma_i) & i = j \\ -\sigma_i\sigma_j & i \neq j \end{cases}$$

Raising one score raises its own probability and lowers every other one, because the total must stay 1.

WORKED EXAMPLE: SOFTMAX OF $\mathbf{Z} = (2.0, 1.0, 0.1)$ (SLIDE 91)

- 1 Exponentials: $e^{2.0} = 7.389$, $e^{1.0} = 2.718$, $e^{0.1} = 1.105$.
- 2 Sum: $7.389 + 2.718 + 1.105 = 11.212$.
- 3 Divide: $\sigma_1 = 7.389/11.212 = 0.659$; $\sigma_2 = 2.718/11.212 = 0.242$; $\sigma_3 = 1.105/11.212 = 0.099$.
- 4 Check: $0.659 + 0.242 + 0.099 = 1.000$. Class 1 wins with 65.9%.

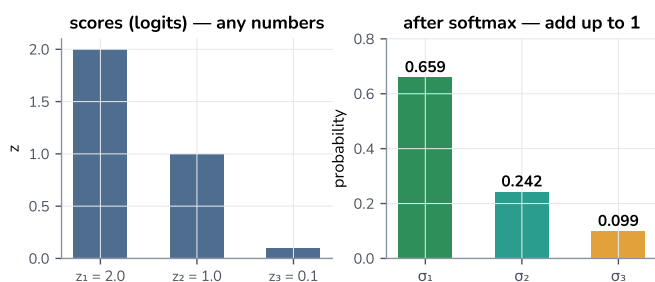


Figure 6.4 Scores in, probabilities out. The order is kept, the gaps are stretched by the exponential, and the total is 1.

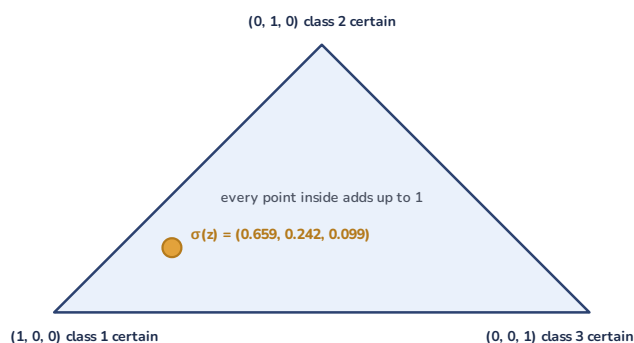


Figure 6.5 The slide's 2-simplex. Every softmax output for three classes is a point in this triangle; ours sits near the "class 1" corner.

? STOP AND THINK

Ravi says: "Why not just pick the biggest score and skip softmax?"

✓ OUR ANSWER

At test time you can. But for training you need a loss that can be differentiated, and cross-entropy needs probabilities. Softmax also tells you *how sure* the network is: 0.659 is a different situation from 0.99, and "Asha 0.51, cat 0.49" is a warning that argmax alone would hide.

📄 ON YOUR SLIDES

Slide 85: FC definition and roles. Slide 86: $\mathbf{y} = f(\mathbf{W}\mathbf{x} + \mathbf{b})$ with $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{W} \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$; total $n \times m + m = m(n + 1)$. Slide 87: the numerical example ($z = (1.6, -0.6)$, $y = (1.6, 0.0)$, 8 parameters). Slide 88: advantages, disadvantages, GAP and 1×1 as alternatives. Slides 89–90: softmax definition, derivative cases, 2-simplex. Slide 91: $(2.0, 1.0, 0.1) \rightarrow (0.659, 0.242, 0.099)$.

! CAREFUL

The first FC layer's input size is fixed by the conv part ($H \cdot W \cdot C$ of the last maps). Change the picture size and that number changes, and the FC layer no longer fits. That is the "fixed input size" disadvantage, and one reason modern networks use global average pooling instead.

✓ CHECK YOURSELF

1. Flatten $7 \times 7 \times 512$. Length of the vector? $7 \cdot 7 \cdot 512 = 25,088$.
2. An FC layer from 400 inputs to 120 outputs: how many parameters? $120 \times (400 + 1) = 48,120$.
3. In $\mathbf{y} = f(\mathbf{W}\mathbf{x} + \mathbf{b})$, what is the shape of $\mathbf{W}$? $m \times n$: **outputs by inputs**.
4. What is $\partial\sigma_1/\partial z_2$? $-\sigma_1\sigma_2$.
5. Softmax outputs $(0.2, 0.3, ?)$. The missing value? **0.5; they must add to 1**.

🕒 IN ONE BREATH

- Flatten: $H \times W \times C \rightarrow H \cdot W \cdot C$ numbers. FC: $\mathbf{y} = f(\mathbf{W}\mathbf{x} + \mathbf{b})$, $m(n + 1)$ parameters.
- Example: $(2, -1, 3) \rightarrow z = (1.6, -0.6) \rightarrow \text{ReLU} \rightarrow (1.6, 0)$; 8 parameters.
- FC layers are heavy and need a fixed input size; GAP and 1×1 convolutions are the modern alternatives.
- Softmax: $e^{z_i} / \sum_j e^{z_j}$; $(2.0, 1.0, 0.1) \rightarrow (0.659, 0.242, 0.099)$; derivative $\sigma_i(1 - \sigma_i)$ or $-\sigma_i\sigma_j$.

Counting knobs and designing eyes

Count a layer's parameters, then design with five patterns and a few rules of thumb.

On your slides: **Module 6, slides 93–108** · Typical architecture · Parameters in conv and pooling layers · Five design patterns · Choosing depth · Practical guidelines · Depth vs width · EfficientNet · Design decision framework

STORY

Asha adds up Nova's knobs on the whiteboard. The first conv layer: a few thousand. The second: a few hundred thousand. The first fully connected layer: millions. "Almost all of it is in the head," she says. Ravi asks the other question: "How many layers should Nova have? Ten? A hundred?" Asha does not know yet. She needs a way to count, and a way to choose.

BIG IDEA

A conv layer has $(K \times K \times C_{in} + 1) \times C_{out}$ parameters: independent of the image size, quadratic in the kernel size, linear in the number of filters. Pooling has none. Good networks reuse five patterns: shrink space while growing channels, repeat one block, add skip connections, squeeze with 1×1 bottlenecks, and look at several scales at once. Depth is chosen by the task, the data size and the compute budget: start shallow, deepen if you underfit.

THINK OF IT LIKE...

Building a tall shelf from one kind of bracket. You do not design every shelf from scratch; you pick a bracket that works and repeat it, adding a diagonal strut (a skip connection) so the tower does not wobble as it gets taller. How tall? As tall as the wall allows (compute), as tall as you have books to fill it (data), and no taller than the job needs (task).

Counting parameters

Slide 93 repeats the typical architecture of Chapter 2 (Figure 2.5): CONV → POOL → CONV → POOL → FC → OUTPUT, spatial size decreasing and depth increasing. Slide 94 gives the count for one conv layer with kernel $K \times K$, C_{in} input channels, C_{out} filters and a bias flag (1 if a bias is used, 0 if not):

$$\text{Total parameters} = (K \times K \times C_{in} + \text{bias}) \times C_{out}$$

Four facts follow, all on the slide. The count is **independent of the input's height and width** (the same filter slides everywhere). Each filter has the same depth as the input. Parameters grow **quadratically** with kernel size ($3 \rightarrow 5$ is $9 \rightarrow 25$, almost $3\times$). Adding filters grows them **linearly**.

WORKED EXAMPLE: THE SLIDE'S TWO LAYERS (SLIDE 95), AND TWO POOLING LAYERS (SLIDE 97)

- 1 First conv layer: RGB input $224 \times 224 \times 3$, kernel 5×5 , 64 filters, bias. Per filter: $5 \times 5 \times 3 + 1 = 76$. Total: $76 \times 64 = 4,864$.
the 224 never appears

- 2 Deeper conv layer: input $56 \times 56 \times 128$, kernel 3×3 , 256 filters, bias. Per filter: $3 \times 3 \times 128 + 1 = 1,153$. Total: $1,153 \times 256 = 295,168$.
smaller kernel, but far more channels in and out
- 3 Max pooling 2×2 , stride 2 on $56 \times 56 \times 128$: output $28 \times 28 \times 128$. Parameters: 0.
- 4 Average pooling, same setting: output $28 \times 28 \times 128$. Parameters: 0.
pooling is a fixed rule (max or mean); its only settings are the hyperparameters k and s

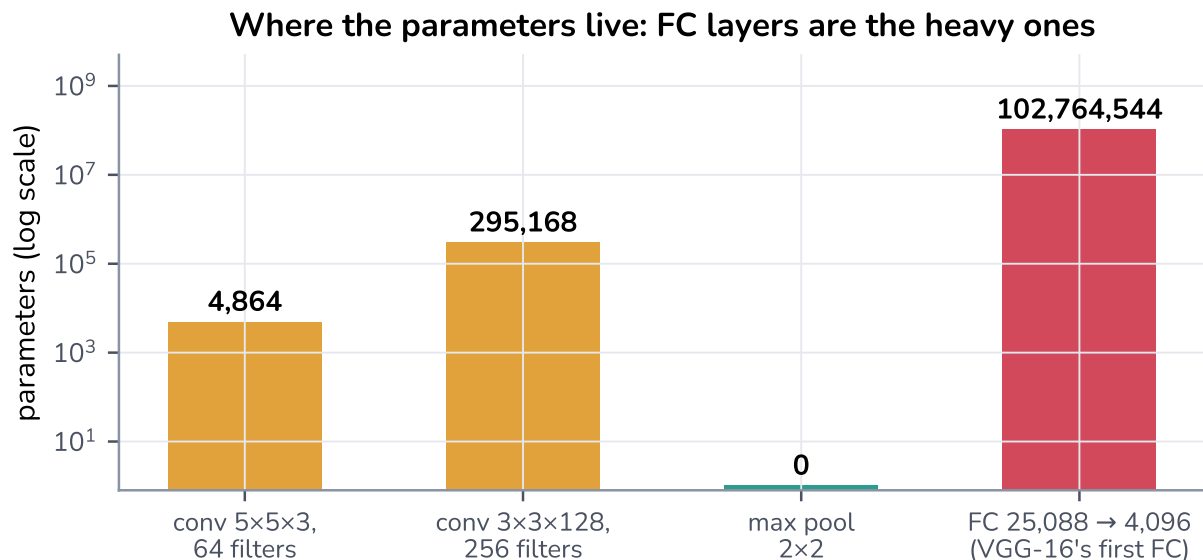


Figure 7.1 Parameter counts on a log scale. The two conv layers from the slide, a pooling layer (zero), and for comparison VGG-16's first FC layer (25,088 inputs $\times$ 4,096 outputs, over 100 million). Conv layers are cheap in parameters; the FC head is where the weight goes.

Five design patterns

Slide 98 asks how to design an effective deep CNN, and lists five patterns common to the successful ones. The goal is a balance between capacity (expressiveness), computation, memory, and training stability.

PATTERN 1: SPATIAL REDUCTION + CHANNEL EXPANSION

As you go deeper, halve the height and width and double the channels: $H \times W \times C \rightarrow \frac{H}{2} \times \frac{W}{2} \times 2C \rightarrow \frac{H}{4} \times \frac{W}{4} \times 4C \rightarrow \dots$ Early layers keep high resolution for fine detail; later layers keep many channels for meaning; the compute per layer stays balanced ($H \times W \times C \approx \text{constant}$, in the slide's words). Done with pooling or with strided convolutions, "typically doubling channels when halving spatial dimensions". Figure 5.2 is this pattern drawn as volumes.

PATTERN 2: REPETITIVE BUILDING BLOCKS

Instead of designing every layer by hand, define one block and repeat it. A **VGG block** is a few 3×3 convs followed by a max pool. A **ResNet block** is two 3×3 convs plus a skip connection. Benefits: modularity, easy to scale up, proven designs.

PATTERN 3: SKIP (RESIDUAL) CONNECTIONS

Problem: deep networks suffer from vanishing gradients and get hard to train. Solution: add an identity shortcut so the block computes $y = F(x) + x$, where $F(x)$ is what the weight layers add. Benefits: easier gradient flow, the network can learn the identity if it needs to, 100+ layers become trainable. Used in ResNet, DenseNet and U-Net.

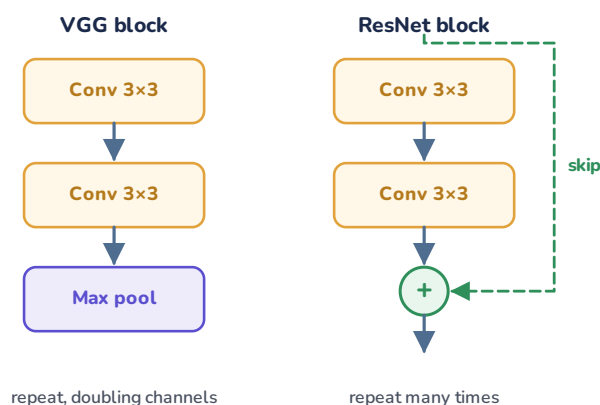


Figure 7.2 The two most-copied blocks: VGG (conv, conv, pool) and ResNet (conv, conv, add the input back).

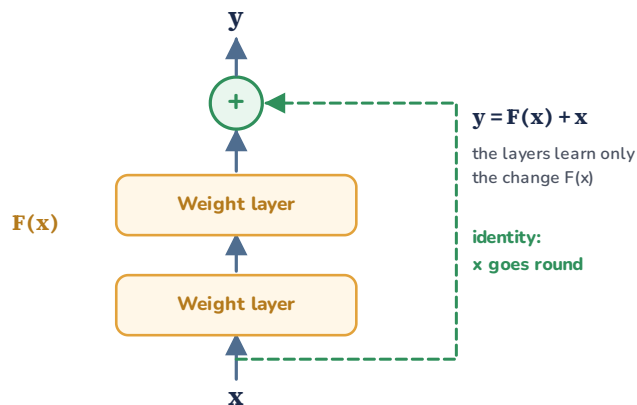


Figure 7.3 A skip connection: x goes around the weight layers and is added back, $y = F(x) + x$. Why this trains so well is explained in Chapter 8.

PATTERN 4: BOTTLENECK DESIGNS

Goal: cut the computational cost without losing performance. Strategy: use 1×1 convolutions to reduce channels before the expensive operation, then expand again. Slide 102's numbers: a 3×3 conv from 256 to 256 channels costs $256 \times 256 \times 9 \approx 590\text{K}$ parameters; the bottleneck 1×1 ($256 \rightarrow 64$), 3×3 ($64 \rightarrow 64$), 1×1 ($64 \rightarrow 256$) costs $256 \cdot 64 + 64 \cdot 64 \cdot 9 + 64 \cdot 256 = 16,384 + 36,864 + 16,384 \approx 70\text{K}$: about 8× fewer. Used in ResNet-50/101/152, Inception and MobileNet. (Chapter 8, Figure 8.10 draws it.)

PATTERN 5: MULTI-SCALE FEATURE EXTRACTION

Process the input at several scales at the same time, then stack the results. The **Inception module** runs a 1×1 , a 3×3 , a 5×5 and a pooling path side by side and concatenates their maps. Benefits: features at different scales, robustness to object size, and the network itself decides which scale matters. Why it needs 1×1 reductions to be affordable is shown in Chapter 8.

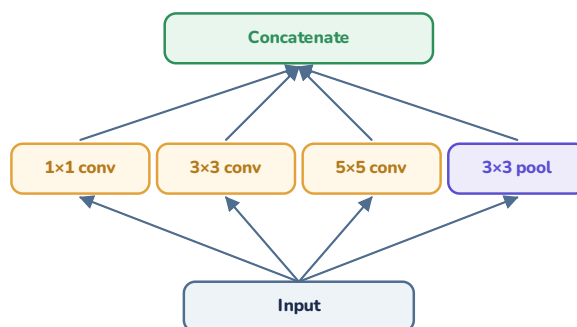


Figure 7.4 The naive Inception module: four parallel views of the same input, joined by concatenation.

How deep?

Slide 104 gives three considerations. **Task complexity**: simple (MNIST) needs 5–10 layers; complex (ImageNet) 50–152; very complex (medical imaging) 100+. **Dataset size**: small (under 10K examples) → shallow, to avoid overfitting; medium (10K–100K) → moderate depth (20–50 layers); large (over 1M) → very deep (50–200). **Computational budget**: mobile/edge → MobileNet, EfficientNet (10–20 layers); desktop/server → ResNet-50, VGG-16 (16–50); data centre → ResNet-152, EfficientNet-B7 (100+). The rule at the bottom of the slide: *start shallow, add depth if underfitting and you have enough data.*

Network	Layers	Params	Use case
LeNet-5	5	60K	MNIST, simple tasks
AlexNet	8	60M	ImageNet (2012)
VGG-16	16	138M	High accuracy, features
ResNet-50	50	25M	General purpose
ResNet-152	152	60M	State-of-the-art accuracy
EfficientNet-B0	18	5M	Mobile, efficient

Slide 105's trends: modern architectures have 50–200 layers with skip connections; efficient ones 10–30 layers with smart design; depth alone is not enough, you need proper training techniques (batch normalisation, skip connections, careful initialisation, learning-rate scheduling). Notice that more parameters is not the same as deeper: VGG-16 has 138M with 16 layers; ResNet-152 has 60M with 152.

DEPTH VS WIDTH

Depth is the number of layers (stacking); width is the number of channels per layer (widening). Slide 106 compares them:

Aspect	Deeper (more layers)	Wider (more channels)
Feature complexity	Higher	Moderate
Receptive field	Larger	Same
Parameters	Sub-linear growth	Quadratic growth
Training difficulty	Harder (vanishing gradients)	Easier
Overfitting risk	Lower (if regularised)	Higher
Inference speed	Slower (sequential)	Faster (parallel)

Examples: ResNet-50 is deep (50 layers) with moderate width (256–2048 channels); Wide ResNet has fewer layers (16–28) but up to 10× the channels. Current consensus: both matter, and the best ratio depends on the task. Slide 107's recommendations: go **deeper** for large diverse datasets, complex tasks that need hierarchical features, when you can use skip connections, and when you have the compute. Go **wider** for smaller, more uniform datasets, when you need speed, when you are limited to shallow networks, or for ensemble-like behaviour (different filters learn different patterns). The hybrid, **EfficientNet**, scales depth, width and input resolution together with **compound scaling**: $\text{depth}^d \times \text{width}^w \times \text{resolution}^r = 2$, a fixed budget split three ways, which gives a better accuracy–efficiency trade-off.

A DESIGN DECISION FRAMEWORK (SLIDE 108)

1. **Start with a baseline:** a proven architecture (ResNet, EfficientNet); use transfer learning if you can (Chapter 9).
2. **Estimate requirements:** dataset size sets a depth limit; the compute budget sets the width/depth trade-off; the input resolution sets the receptive field you need.
3. **Iterate systematically:** underfitting → add depth or width; overfitting → regularise (dropout, data augmentation) before shrinking the model; too slow → reduce width or use efficient blocks.
4. **Apply the patterns:** always spatial reduction + channel expansion; deep networks need skip connections; tight budgets need bottlenecks.

? STOP AND THINK

Ravi says: "If parameters do not depend on the image size, can Nova use huge pictures for free?"

✓ OUR ANSWER

The parameter count is free; the work is not. Every filter is applied at every position, so compute and memory grow with $H \times W$ (Chapter 5: $\text{memory} \propto H \times W \times C$). And the first FC layer's input, $H \cdot W \cdot C$ of the last maps, does grow with the picture. Bigger pictures cost time and memory, not weights.

📄 ON YOUR SLIDES

Slide 93: typical CNN architecture. Slide 94: $(K \times K \times C_{in} + \text{bias}) \times C_{out}$ and its four properties. Slide 95: $76 \rightarrow 4,864$ and $1,153 \rightarrow 295,168$. Slides 96–97: pooling has zero parameters, two examples. Slides 98–103: the five patterns. Slide 104: how to choose depth. Slide 105: the practical table and trends. Slides 106–107: depth vs width, recommendations, compound scaling. Slide 108: the design decision framework.

! CAREFUL

"Bias: yes/no" changes the count. With bias, per filter $K \times K \times C_{in} + 1$; without, drop the $+1$. Exam questions usually say which. And the count is per *filter* times the number of filters; do not forget the $\times C_{out}$.

✓ CHECK YOURSELF

1. Conv layer: 7×7 kernel, 3 input channels, 64 filters, bias. Parameters? $(49 \times 3 + 1) \times 64 = 148 \times 64 = 9,472$.
2. Does doubling the image size change a conv layer's parameter count? **No; it changes compute and memory.**
3. Name the five design patterns. **Spatial reduction + channel expansion; repetitive blocks; skip connections; bottlenecks; multi-scale extraction.**
4. Small dataset, 5,000 images: deep or shallow? **Shallow (avoid overfitting), or transfer learning.**
5. Which grows parameters faster, adding channels or adding layers? **Adding channels (quadratic) vs layers (sub-linear).**

🔥 IN ONE BREATH

- Conv parameters = $(K \cdot K \cdot C_{in} + 1) C_{out}$: no H , no W . Pooling: 0. FC layers hold most weights.
- Five patterns: shrink + widen; repeat a block; skips $y = F(x) + x$; 1×1 bottlenecks (590K $\rightarrow$ 70K); multi-scale.
- Depth follows task, data size and budget: start shallow, deepen if underfitting. Deeper sees further; wider is faster.

The famous ones

LeNet, AlexNet, VGG, NiN, GoogLeNet and ResNet: six networks, six ideas.

On your slides: **Module 6, slides 110–135** · LeNet-5 · AlexNet · VGGNet · Network in Network · GoogLeNet / Inception · ResNet: degradation, residual learning, blocks, family, why it works

STORY

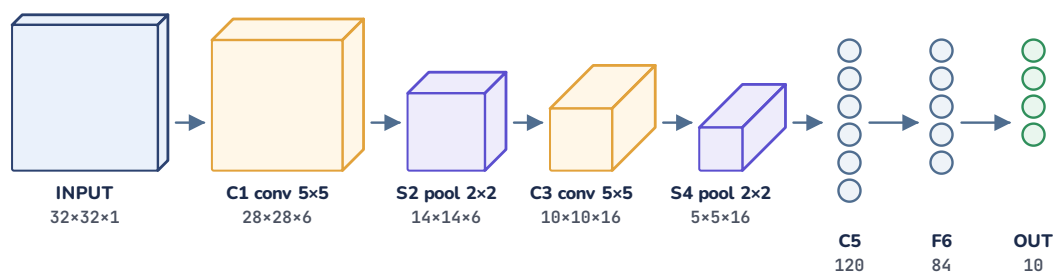
Asha pins a family tree of networks above her desk: at the top, a 1998 network that read handwritten postcodes; at the bottom, a 2015 network 152 layers deep that beat people at naming a thousand kinds of object. "Each one fixed one problem the one before it had," she tells Ravi. "Know the six problems, and you know why Nova looks the way it does."

BIG IDEA

The history of CNNs is a short list of fixes. LeNet set the recipe (conv → pool → conv → pool → FC). AlexNet made it work at scale with ReLU, dropout, data augmentation and GPUs. VGG showed that stacks of 3×3 filters beat big filters. NiN replaced heavy FC layers with 1×1 convolutions and global average pooling. GoogLeNet looked at several scales at once and used 1×1 bottlenecks to afford it. ResNet added skip connections so that 100+ layers could be trained at all.

LeNet-5 (1998)

Built for handwritten digit recognition (MNIST). Slide 110: INPUT → CONV → POOL → CONV → POOL → FC → FC → OUTPUT; the first successful CNN; sigmoid/tanh activations; "subsampling" layers (average pooling); about 60K parameters; 32×32 grayscale input. Slide 111 is the original figure: C1 6 maps of 28×28 , S2 6 of 14×14 , C3 16 of 10×10 , S4 16 of 5×5 , C5 120, F6 84, output 10: exactly the sizes you computed in Chapter 2.

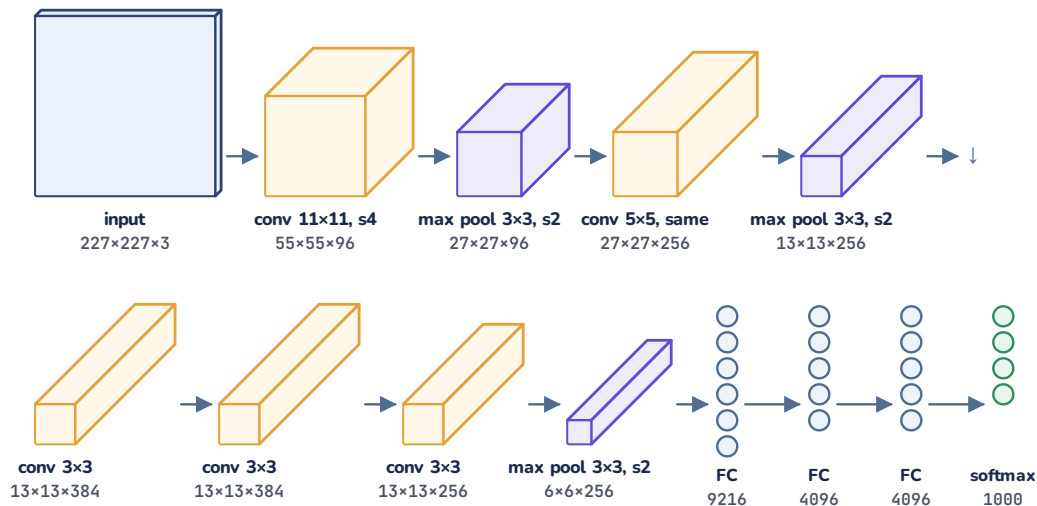


1998 · sigmoid/tanh activations · average pooling ("subsampling") · about 60K parameters

Figure 8.1 LeNet-5. Two rounds of 5×5 convolution and 2×2 subsampling, then three fully connected stages. Every later network is a bigger, better-trained version of this shape.

AlexNet (2012)

Slide 112's innovations: **ReLU** activations instead of sigmoid/tanh (faster training, no saturation); **dropout** for regularisation; **data augmentation**; **GPU** acceleration (it was trained on two GPUs); and Local Response Normalisation (LRN, a normalisation trick that has since been replaced by batch norm). Architecture: 8 layers, 5 conv + 3 FC; 60M parameters; input $224 \times 224 \times 3$. It won ImageNet 2012 with a top-5 error of 15.3%, far ahead of everything else, and "sparked the deep learning revolution".



2012 · 5 conv + 3 FC = 8 layers · ReLU · dropout · trained on two GPUs · about 60M parameters

Figure 8.2 AlexNet as drawn on slide 113 (with the 227×227 input that makes the arithmetic work: $(227 - 11)/4 + 1 = 55$). Big 11×11 filters with stride 4 at the start, three max pools, then two FC layers of 4,096 and a softmax over 1,000 classes. The last maps, $6 \times 6 \times 256 = 9,216$ numbers, feed the first FC layer.

VGGNet (2014)

Slide 114's principles: very small (3×3) filters only; deep (16–19 layers); simple and uniform: only 3×3 convs (stride 1, pad 1, so "same" size) and 2×2 max pools (stride 2) throughout. VGG-16 has 13 conv layers + 3 FC layers. Advantages: a simple homogeneous design, two 3×3 convs cover the same patch as one 5×5 with fewer parameters, and more non-linearities.

WORKED EXAMPLE: TWO 3×3 VS ONE 5×5

Both from C channels to C channels, biases ignored.

- 1 One 5×5 layer: $5 \times 5 \times C \times C = 25C^2$ weights.
- 2 Two 3×3 layers: $9C^2 + 9C^2 = 18C^2$ weights.
 $18C^2 < 25C^2$: 28% fewer
- 3 Receptive field of the stack: $3 + (3 - 1) = 5$. Same 5×5 view.
- 4 Bonus: a ReLU between the two 3×3 layers; the 5×5 has none inside.

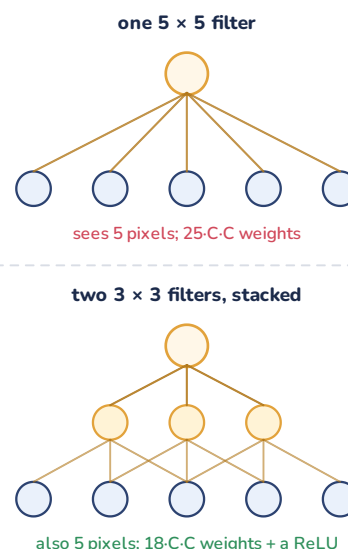


Figure 8.3 One 5-wide filter and two stacked 3-wide filters see the same five inputs. The stack is cheaper and has an extra ReLU.

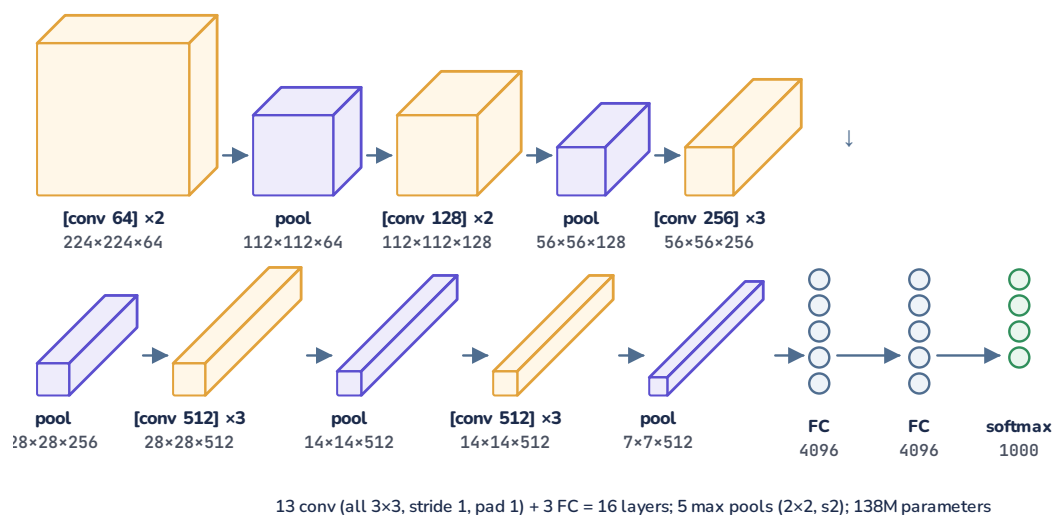


Figure 8.4 VGG-16 from slide 115. Five stages, each "a few 3×3 convs, then a pool"; channels $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512$ as the maps go $224 \rightarrow 7$. The three FC layers at the end hold most of its 138M parameters.

Network in Network (2013)

Slide 116's core innovation: replace the linear filter with a "micro neural network" applied to each patch, called an **MLPconv** layer, to get more expressive power with fewer parameters. Key ideas: MLPconv (a mini multi-layer perceptron instead of a linear filter); 1×1 convolutions, which add non-linearity and mix channels; **global average pooling** to replace the FC layers; and fewer parameters, so less overfitting. Slide 117 writes the two side by side. Ordinary convolution on a patch $\mathbf{x}$: $y = \max(0, \mathbf{w}^\top \mathbf{x} + b)$. MLPconv: $h_1 = \max(0, \mathbf{w}_1^\top \mathbf{x} + b_1)$, $h_2 = \max(0, \mathbf{w}_2^\top h_1 + b_2)$, $y = \max(0, \mathbf{w}_3^\top h_2 + b_3)$: a conv, then two 1×1 convs, each with a ReLU.

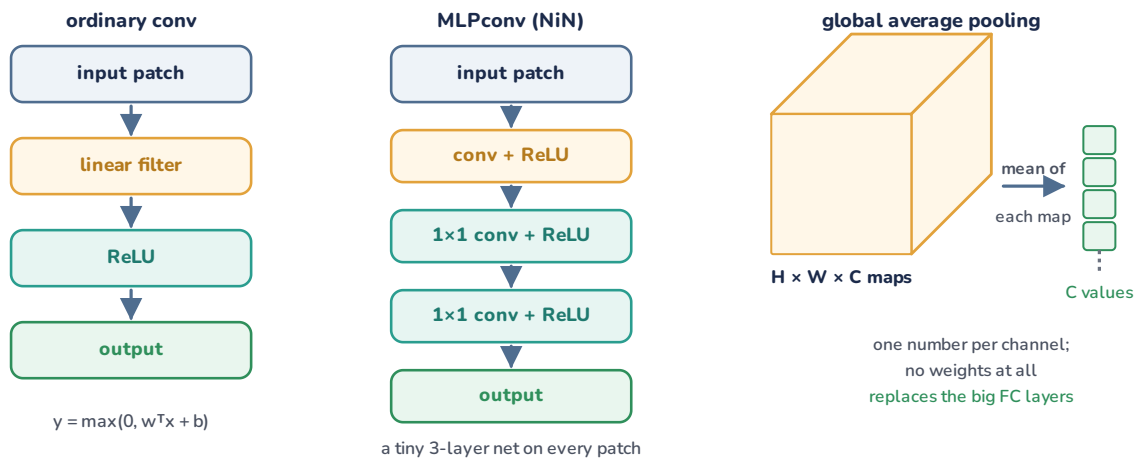


Figure 8.5 Left: an ordinary conv is one linear filter and a ReLU. Middle: an MLPconv block is a conv followed by two 1×1 convs, each with ReLU. Right: global average pooling turns each of the C final maps into its mean, giving C numbers with no weights at all.

Slide 118's architecture. A NiN block = one conv (5×5 or 3×3 , the "spatial" step) + two 1×1 convs (the "channel" steps). The full network on $224 \times 224 \times 3$:

Layer	Configuration	Output size
NiN block 1	11×11 conv, 96 filters (stride 4)	$54 \times 54 \times 96$
Max pool	3×3 , stride 2	$26 \times 26 \times 96$
NiN block 2	5×5 conv, 256 filters	$26 \times 26 \times 256$
Max pool	3×3 , stride 2	$12 \times 12 \times 256$
NiN block 3	3×3 conv, 384 filters	$12 \times 12 \times 384$
Max pool	3×3 , stride 2	$5 \times 5 \times 384$
NiN block 4	3×3 conv, 1000 filters	$5 \times 5 \times 1000$
Global average pool	average each channel	$1 \times 1 \times 1000$
Softmax	classification	1000 classes

Slide 119, global average pooling: FC layers have millions of parameters (overfitting), need a fixed input size, and lose spatial correspondence. GAP averages each map instead, $\text{Output}_c = \frac{1}{H \times W} \sum_i \sum_j \text{FeatureMap}_c(i, j)$: no parameters, more robust. The last block has exactly 1000 filters, one per class, so GAP gives one score per class. Slide 120's impact: 1×1 convs in Inception and ResNet bottlenecks, MobileNet built on them, GAP standard instead of FC.

GoogLeNet / Inception (2014)

Slide 121: objects appear at different scales, and it is hard to choose a filter size (1×1 ? 3×3 ? 5×5 ?). The Inception module's answer: use all of them in parallel and concatenate. Winner of ImageNet 2014 with 6.67% top-5 error; 22 layers deep; only 5M parameters ($12\times$ fewer than AlexNet); a stack of Inception modules. The naive module (slide 122, Figure 7.4) runs 1×1 , 3×3 , 5×5 and a 3×3 max pool side by side, but with 256 input channels and 256 outputs per path the computation "explodes". Slide 123's fix: a 1×1 convolution *before* the 3×3 and 5×5 to cut channels, and one *after* the pool.

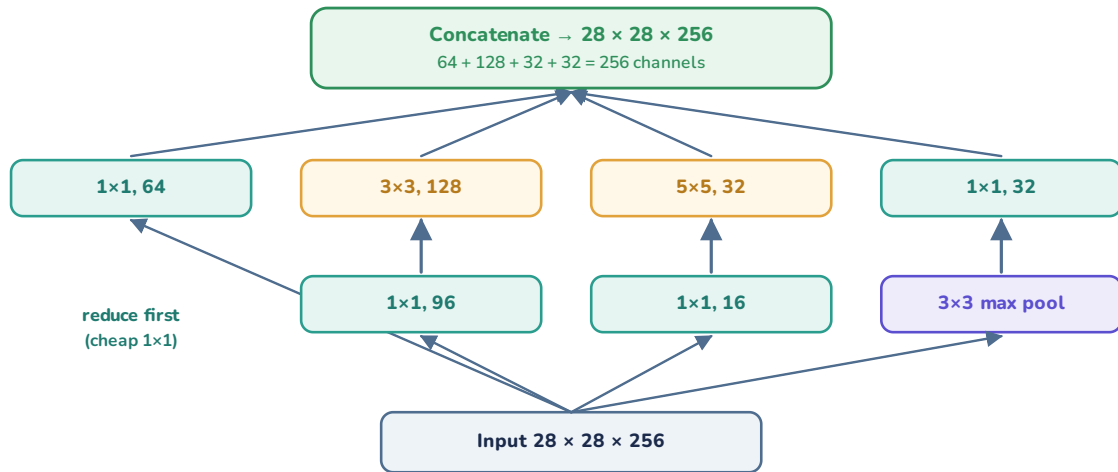


Figure 8.6 The Inception module with dimensionality reduction on a $28 \times 28 \times 256$ input. The 3×3 path first squeezes to 96 channels, the 5×5 path to 16; the pooling path is squeezed afterwards. The four outputs, $64 + 128 + 32 + 32 = 256$ channels, are stacked into $28 \times 28 \times 256$.

WORKED EXAMPLE: THE COMPUTATIONAL SAVING (SLIDE 124)

Count multiply-adds as $(\text{output height} \times \text{width}) \times (\text{input channels}) \times (\text{output channels}) \times (\text{kernel area})$.

- 1 Naive 3×3 , $256 \rightarrow 128$: $28 \times 28 \times 256 \times 128 \times 9 \approx 230\text{M}$.
- 2 Naive 5×5 , $256 \rightarrow 32$: $28 \times 28 \times 256 \times 32 \times 25 \approx 160\text{M}$. Naive total: about 390M.
- 3 With reduction. 1×1 , $256 \rightarrow 96$: $28 \times 28 \times 256 \times 96 \approx 19\text{M}$. Then 3×3 , $96 \rightarrow 128$: $28 \times 28 \times 96 \times 128 \times 9 \approx 86\text{M}$.
- 4 1×1 , $256 \rightarrow 16$: $\approx 3\text{M}$. Then 5×5 , $16 \rightarrow 32$: $28 \times 28 \times 16 \times 32 \times 25 \approx 10\text{M}$. Reduced total: about 118M.
- 5 $390/118 \approx 3.3$: a 3.3× reduction. "This is the bottleneck principle that became standard in deep networks."

Slide 125, the whole 22-layer network: 7×7 stride-2 conv ($112 \times 112 \times 64$) $\rightarrow$ max pool ($56 \times 56 \times 64$) $\rightarrow 3 \times 3$ conv ($56 \times 56 \times 192$) $\rightarrow$ max pool ($28 \times 28 \times 192$) $\rightarrow$ Inception 3a, 3b ($28 \times 28 \times 256, 480$) $\rightarrow$ max pool ($14 \times 14 \times 480$) $\rightarrow$ Inception 4a–4e ($14 \times 14 \times 512$ to 832) $\rightarrow$ max pool ($7 \times 7 \times 832$) $\rightarrow$ Inception 5a, 5b ($7 \times 7 \times 832$ to 1024) $\rightarrow$ global average pool ($1 \times 1 \times 1024$, "NiN style") $\rightarrow$ dropout 40% $\rightarrow$ FC + softmax, 1000 classes. Nine Inception modules in all. Slide 126, why it works: multi-scale extraction; sparse connections; efficient computation through 1×1 bottlenecks; more depth and width without a computational explosion. **Auxiliary classifiers**: extra softmax heads on middle layers during training, to fight vanishing gradients; removed at test time.

Slide 127, the variants: v1 (GoogLeNet, 2014); v2 (2015, batch normalisation); v3 (2015, factorised convolutions); v4 (2016, with residual connections); Inception-ResNet (2016, the hybrid). The factorisation arithmetic: a 5×5 conv costs $25n^2$ operations, two 3×3 convs $9n^2 + 9n^2 = 18n^2$ (28% fewer, same receptive field); a 3×3 ($9n^2$) becomes $3 \times 1 + 1 \times 3$ ($3n^2 + 3n^2 = 6n^2$).

ResNet (2015)

Slide 128's surprising discovery: deeper plain networks did *worse* than shallow ones, even on the training set. A 56-layer network had a higher training error than an 18-layer one. That is not overfitting (overfitting shows up on test data, not training data); it is an **optimisation** problem. The hypothesis: the deeper network could not even learn to copy its input through the extra layers, that is, it struggled to learn an identity mapping.

≈ THINK OF IT LIKE...

Editing a draft versus rewriting from a blank page. If the draft is already good, an editor only writes the small changes; rewriting from scratch, she must first reproduce everything that was fine, and can easily make it worse. A residual block is an editor: it keeps the input and learns only the change.

Slide 129's key insight: instead of learning the target mapping $H(x)$ directly, learn the **residual** $F(x) = H(x) - x$, and add x back: $H(x) = F(x) + x$. Why is this easier? If the best thing a block can do is nothing (identity), it only needs $F(x) = 0$, which means pushing weights to zero. That is easy. Learning an exact identity through several conv layers is hard.

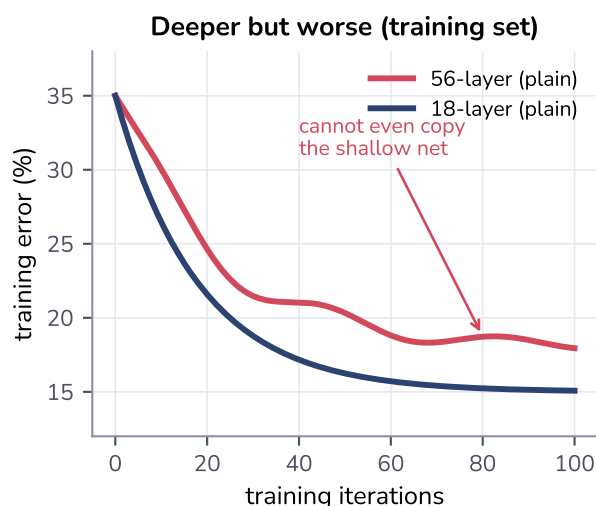


Figure 8.7 The degradation problem (schematic of slide 128): the 56-layer plain network trains worse than the 18-layer one.

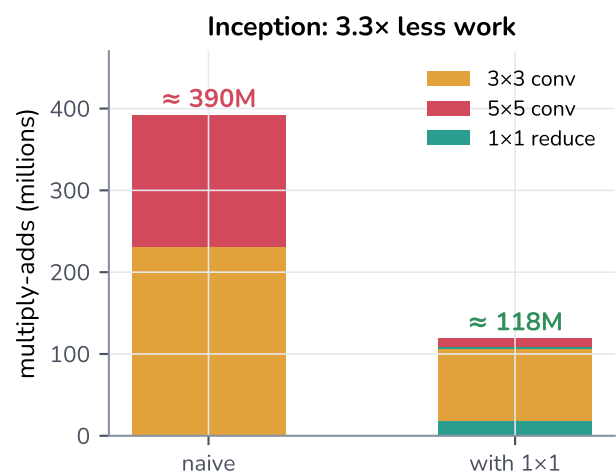


Figure 8.8 The Inception saving from the worked example: about 390M multiply-adds naive, about 118M with 1×1 reductions.

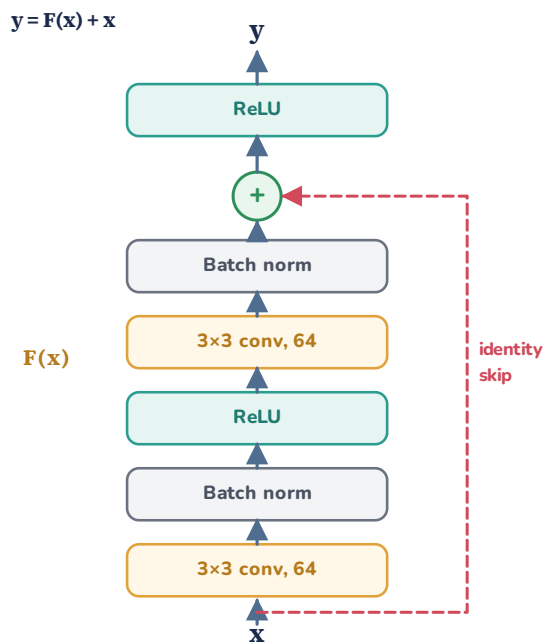


Figure 8.9 The basic residual block (slide 130; ResNet-18/34): two 3×3 convs with batch norm after each, ReLU in the middle and after the addition, and an identity skip.

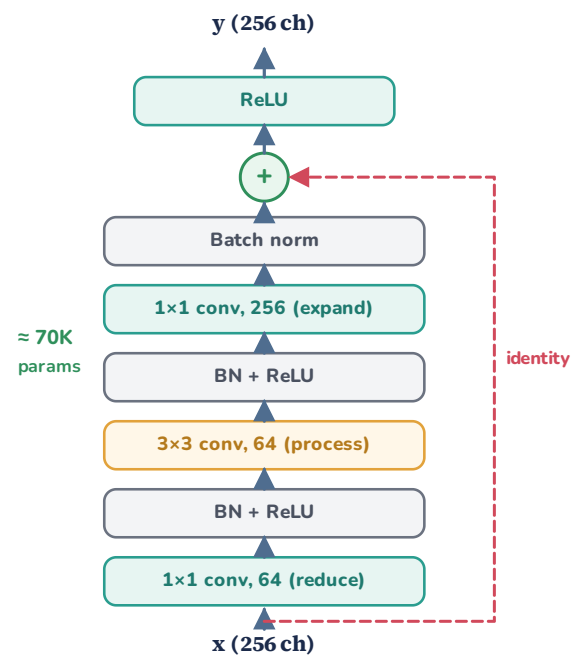


Figure 8.10 The bottleneck block (slide 131; ResNet-50/101/152): 1×1 reduce ($256 \rightarrow 64$), 3×3 process, 1×1 expand ($64 \rightarrow 256$). $256 \cdot 64 + 64 \cdot 64 \cdot 9 + 64 \cdot 256 \approx 70\text{K}$ vs $256 \cdot 256 \cdot 9 \approx 590\text{K}$: 8.4 $\times$ fewer.

Slide 132: what if the block changes the size, say $56 \times 56 \times 64$ in and $28 \times 28 \times 128$ out? Then x cannot simply be added to $F(x)$. The fix is a **projection shortcut**: a 1×1 convolution with stride 2 on the skip path, which halves the spatial size and changes the channels, so the shapes match.

Slide 133, the family. All start with a 7×7 , 64-filter, stride-2 conv, then four stages of blocks. ResNet-18 and 34 use basic blocks $[3 \times 3, 64] \times 2$ and so on; ResNet-50 and 101 use bottleneck blocks $[1 \times 1, 64; 3 \times 3, 64; 1 \times 1, 256] \times 3$ and so on, with the third stage repeated 6 times in ResNet-50 and 23 times in ResNet-101.

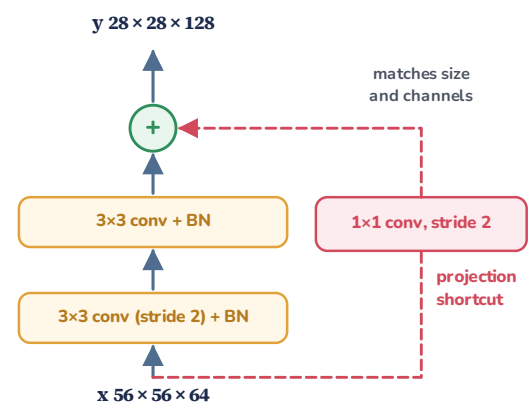


Figure 8.11 A projection shortcut: the skip path gets its own 1×1 , stride-2 conv so that x can be added to the smaller, deeper $F(x)$.

	ResNet-18	ResNet-34	ResNet-50	ResNet-101
Block type	basic	basic	bottleneck	bottleneck
Blocks per stage	2, 2, 2, 2	3, 4, 6, 3	3, 4, 6, 3	3, 4, 23, 3
Total layers	18	34	50	101
Parameters	11.7M	21.8M	25.6M	44.5M
Top-5 error	10.2%	9.6%	7.7%	7.1%

Σ MATHS, STEP BY STEP: WHY THE SKIP HELPS THE GRADIENT

PLAIN MATHS

A block: $y = F(x) + x$. Differentiate with respect to x :

$$\frac{\partial y}{\partial x} = \frac{\partial F}{\partial x} + 1$$

Chain rule for the loss L :

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \left(1 + \frac{\partial F}{\partial x} \right)$$

Even if $\partial F/\partial x$ is tiny, the "1" carries $\partial L/\partial y$ straight through. That is the **gradient highway**. In a plain network there is no "1", and Book 5's product of small numbers shrinks the gradient to nothing.

TWO MORE VIEWS

Ensemble behaviour. At every block the signal can go through F or skip it: n blocks give 2^n paths of different lengths (three blocks: 8), like an ensemble of shallower networks.

→ **Feature reuse.** Early features pass on unchanged; no information bottleneck; each layer refines features instead of recreating them.

Slide 135: ResNet won ImageNet 2015 with 3.6% error (better than humans on this benchmark) and made 50–152-layer networks trainable (even 1000+). Contributions: residual learning $F(x) = H(x) - x$, identity skips, very deep networks, the 1×1 bottleneck. Skips are now standard everywhere (DenseNet, U-Net, EfficientNet, Transformers); ResNet features are the usual backbone for transfer learning (Chapter 9).

📖 ON YOUR SLIDES

Slides 110–111: LeNet-5. 112–113: AlexNet. 114–115: VGGNet. 116–120: NiN. 121–127: GoogLeNet. 128–135: ResNet.

! CAREFUL

"Deeper was worse" (slide 128) is about *training* error. Test error would call for regularisation; training error calls for an easier optimisation problem, which the skip provides.

✓ CHECK YOURSELF

1. Two 3×3 convs vs one 5×5 : weights? $18C^2$ vs $25C^2$; **same view**.
2. Global average pooling on $H \times W \times C$ maps outputs? C **means**; **zero parameters**.
3. Why a 1×1 conv before Inception's 5×5 ? **Cut 256 → 16 channels: 160M → 10M ops**.
4. The residual block's gradient rule? $\partial L/\partial x = \partial L/\partial y (1 + \partial F/\partial x)$.

🌟 IN ONE BREATH

- LeNet-5 → AlexNet (ReLU, dropout, GPUs) → VGG (all 3×3) → NiN (1×1 , GAP) → GoogLeNet (Inception) → ResNet (skips).
- Two 3×3 = one 5×5 view, $18C^2 < 25C^2$. Inception: 390M → 118M ops. Bottleneck block: 70K vs 590K.
- Skips: $\partial L/\partial x = \partial L/\partial y (1 + \partial F/\partial x)$, a gradient highway; 2^n paths; feature reuse.

Borrowing eyes, and using them

Borrow eyes trained on ImageNet, then see what CNNs are used for and how to make them work.

On your slides: **Module 6, slides 137–152** · Transfer learning: concept, ImageNet, three strategies, guidelines · Applications: classification, detection, segmentation, others, beyond vision · Implementation tips · Pitfalls · Key takeaways · Practical guidelines

📖 STORY

Asha has three hundred photos of herself, forty of the cat, and a corridor of "neither". A network trained from scratch on that memorises the photos and learns nothing. Ravi points at the family tree: "That ResNet saw 1.2 million pictures. Why not use its eyes?" Asha downloads it, cuts off its 1000-way head, and puts on a new 3-way head. By the evening Nova lights up for Asha, lights up for the cat, and stays dark for the corridor.

! BIG IDEA

Transfer learning: use a model trained on one task as the starting point for a related task. The early layers of an ImageNet network (edges, textures, parts) are useful for almost any picture, so keep them. Depending on how much data you have and how similar it is, either freeze them and train only a new head, fine-tune everything gently, or fine-tune only the later layers.

≈ THINK OF IT LIKE...

Hiring a trained photographer for a new job. She already knows light, focus, edges and faces (the conv layers); you only teach her which of your three subjects to shout about (the new head). If the job is unusual, you let her adjust her habits a little (fine-tuning) rather than sending her back to school.

Transfer learning

Slide 137's reasons: **limited data** (small datasets benefit from pre-trained features); **computational efficiency** (no training from scratch); **better performance**; **feature reusability** (low-level features transfer). The common scenario: source task = ImageNet classification (1.2 million images, 1000 classes); target task = your own problem; assumption = similar low-level features. Slides 138–139 show ImageNet: the 96 first-layer kernels AlexNet learned (edges and colour blobs, as Chapter 4 promised) and example predictions. ImageNet accuracy is **top-5**: correct if the true label is among the five most probable guesses.

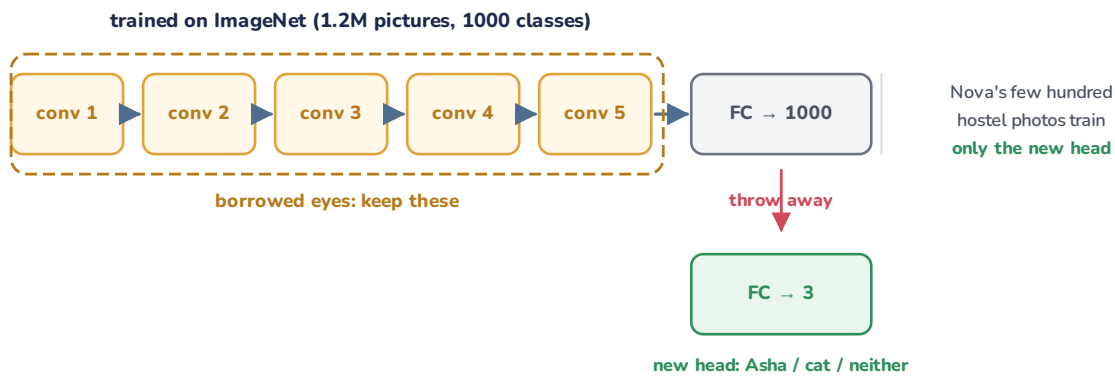


Figure 9.1 Nova's transfer. The conv layers trained on ImageNet are kept ("borrowed eyes"); the 1000-way head is thrown away; a new 3-way head is trained on Asha's few hundred photos. That head is one FC layer, tiny compared with the borrowed part.

Slide 140 gives three strategies.



Figure 9.2 Grey layers are frozen; green layers are trained. Feature extraction: freeze everything, train only the classifier (small dataset, similar data). Fine-tuning: start from the pre-trained weights, train everything with a small learning rate (medium dataset, somewhat similar). Selective fine-tuning: freeze the early layers, train the later ones (medium dataset, different but related).

Slide 141 turns this into a grid with two axes: how *similar* your data is to ImageNet, and how *much* of it you have.

- Similar, small: feature extraction (freeze conv layers, train the classifier only).
- Similar, large: fine-tune all layers with a small learning rate.
- Different, small: train from scratch is risky; try selective fine-tuning.
- Different, large: train from scratch (a large dataset in a different domain).

Nova's photos are ordinary camera pictures, similar to ImageNet, and few: top-left cell, feature extraction. Its new head has $3 \times (2048 + 1) = 6,147$ parameters on a ResNet-50's 2048 features, against 25M borrowed ones.

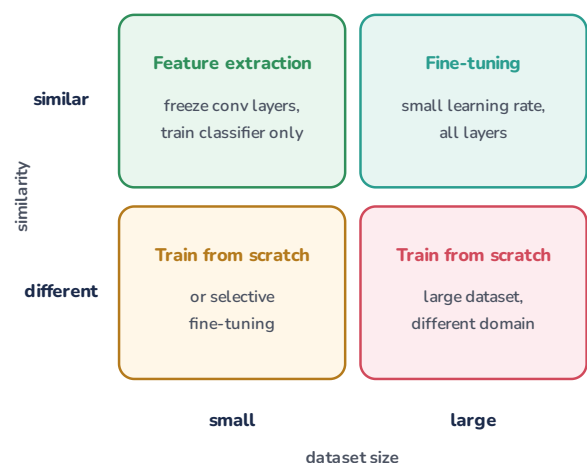


Figure 9.3 The slide's guideline grid: data similarity (up) against dataset size (across).

? STOP AND THINK

Ravi says: "Why a *small* learning rate when fine-tuning? Bigger steps learn faster."

✓ OUR ANSWER

The borrowed weights are already good. A big step would throw them far from where they are, destroying the features you borrowed them for, before the small dataset can teach anything back. Small steps keep the features and only nudge them toward the new task.

What CNNs are used for

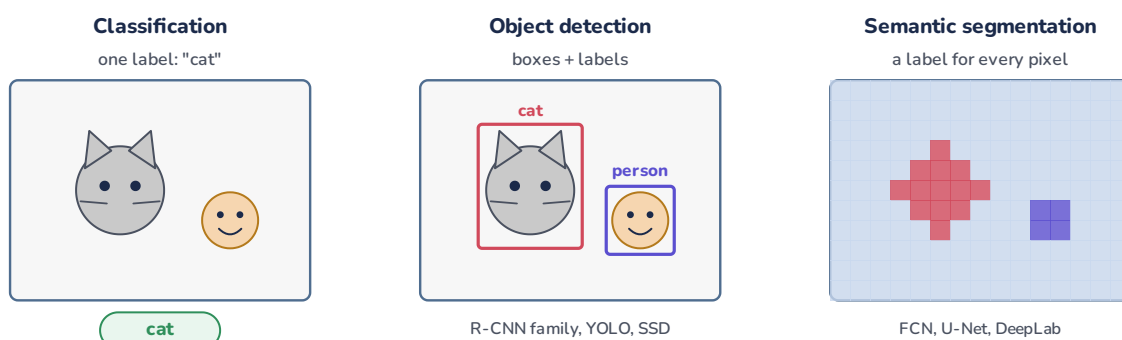


Figure 9.4 Three levels of "where": one label for the whole picture, a box and label per object, a label for every pixel.

- **Image classification** (slide 143): one label per image (CIFAR-10, ImageNet, medical diagnosis). Standard CNN → global average pooling or FC → softmax; cross-entropy loss; accuracy and top- k accuracy. Considerations: augmentation, class imbalance, proper splits.
- **Object detection** (slide 144): locate and classify several objects; output = bounding boxes + labels. **Two-stage**: propose regions, then classify (R-CNN, Fast R-CNN, Faster R-CNN). **One-stage**: boxes and labels in one pass (YOLO, SSD). Challenges: multiple scales, object sizes, real-time needs.
- **Semantic segmentation** (slide 145): classify every pixel. FCN (fully convolutional), U-Net (encoder-decoder with skips), DeepLab (atrous, that is dilated, convolutions, multi-scale). Uses: medical imaging, autonomous driving, satellite imagery.
- **Other vision tasks** (slide 146): face recognition (identity verification, detection), style transfer, image generation (GANs, VAEs), super-resolution.
- **Beyond vision**: text classification and sentiment analysis, audio (speech, music), time series (1-D convolutions), graph neural networks (graph convolutions). Anything on a grid or a line can be convolved.

Making it work: tips and pitfalls

Slide 148, practical tips. **Data**: normalise pixels (0–1 or standardise); augment (rotation, flipping, cropping, colour jitter); proper train/validation/test splits. **Architecture**: start simple, add complexity gradually; batch normalisation for faster training; dropout for regularisation; monitor validation performance. **Training**: a learning-rate schedule; early stopping; gradient clipping; pre-trained models whenever possible.

Pitfall (slide 149)	Symptoms	Solutions
Overfitting	High training accuracy, poor validation accuracy	More data, data augmentation, dropout, L2 regularisation
Underfitting	Poor on both training and validation	Larger model, more training time, less regularisation
Vanishing / exploding gradients	Training stalls or blows up	Proper initialisation, batch normalisation, residual connections
Computational efficiency	Too slow, out of memory	Appropriate batch size, mixed-precision training, model compression

Key takeaways and practical guidelines

Slide 151: CNNs exploit spatial structure through local connectivity and parameter sharing; convolution, pooling and non-linear activations are the building blocks; features are learned hierarchically, edges → shapes → objects. Architectures evolved LeNet → AlexNet → VGG → GoogLeNet → ResNet → modern, with ReLU, dropout, batch normalisation and residual connections as the key innovations. Slide 152: start with transfer learning; augment; monitor training and validation together; regularise. Homework: build a simple CNN, try architectures on CIFAR-10, try transfer learning, explore augmentation.

ON YOUR SLIDES

Slide 137: transfer learning. 138–139: ImageNet. 140–141: strategies and the grid. 143–146: applications. 148–149: tips, pitfalls. 151–152: takeaways.

CAREFUL

Use the *same* preprocessing the pre-trained model was trained with (input size, pixel normalisation): borrowed eyes expect the world to look as it did in training.

CHECK YOURSELF

1. ImageNet size, and top-5 accuracy? **1.2M images, 1000 classes; true label among the top five guesses.**
2. Small dataset, similar to ImageNet: which strategy? **Feature extraction (freeze, train the head).**
3. Name a two-stage and a one-stage detector, and a segmentation model with dilated convolutions. **Faster R-CNN; YOLO or SSD; DeepLab.**
4. High training accuracy, poor validation: what, and what to do? **Overfitting; more data, augmentation, dropout.**

IN ONE BREATH

- Transfer learning: keep the ImageNet conv layers, replace the head; freeze, fine-tune gently, or fine-tune the later layers.
- Classification (one label), detection (boxes: R-CNN family, YOLO/SSD), segmentation (every pixel: FCN, U-Net, DeepLab).
- Normalise, augment, split; batch norm, dropout, early stopping. Nova can hear and see.

Where you are in the series

Six small books, one story. Asha builds Nova, a robot that must learn to hear its name — and along the way she meets every idea in the Deep Neural Network course, in the order the course teaches them.

BOOK 1

Teaching a Machine to Listen

What deep learning is, and the four parts of every learning problem

BOOK 2

One Tiny Brain Cell

The artificial neuron, the perceptron, logic gates and the XOR wall

BOOK 3

Guessing How Much

A linear neuron for regression, squared loss and gradient descent

BOOK 4

Guessing Which One

Sigmoid and softmax neurons, cross-entropy, SGD and mini-batches

BOOK 5

A Real Brain

Hidden layers, activations, backpropagation by hand, and design

BOOK 6

Eyes for the Machine

Convolution, feature maps, pooling, famous CNNs and transfer learning

Nova can hear and see. The rest of the course is about making it wiser.